PHARMACY MANAGEMENT SYSTEM

Object-Oriented Programming - Mini Project Report

Submitted in partial fulfillment of the requirements for the
Bachelor of Technology in Computer Science and Engineering

Project Title	Pharmacy Management System
Subject	Object-Oriented Programming (OOP)
Academic Year	2025 - 2026

Team Members

Name	Register Number	Roll No.
Kenneth Martin	240962047	8
Sam Thomas	240962242	18
Mohammed Rayyan Sheik	240962073	12
Vedank Singh	240962043	7

Introduction

Running a pharmacy involves far more record-keeping than most people realise. Every tablet sold, every prescription received, every new batch delivered from a supplier - all of it needs to be logged somewhere. When that somewhere is a paper notebook, things go wrong quickly. A medicine gets billed at the wrong price. Someone sells a batch that expired two weeks ago. A customer's prescription history disappears when the register gets full and a new one is started. These are not theoretical scenarios - they happen regularly in pharmacies that have not moved away from manual record keeping.

Our group chose to build a desktop application for pharmacy management as our semester project. The application runs on Java with a visual front end built using JavaFX. A staff member can look up any medicine in seconds, raise an invoice with automatic calculations, store patient prescriptions digitally, and see a live warning the moment any medicine drops below the minimum stock level or approaches its expiry date. Two separate background processes keep watch over stock and expiry round the clock, completely independent of whatever the staff member is doing on screen at the time.

The project gave us a concrete reason to use OOP properly rather than just reading about it. Every concept from the syllabus ended up appearing naturally - private fields with controlled access, classes that share behaviour through parent classes, the same method name behaving differently depending on the arguments passed, shared behaviour defined through interfaces, type-independent data storage through generics, and background workers through threads. Nothing felt forced. The structure of a real pharmacy application just happens to need all of these things.

Problem Statement

Pharmacy Management System with Inventory Control and Billing Automation

A typical neighbourhood pharmacy juggles stock from dozens of suppliers, serves hundreds of patients, and processes prescriptions from multiple doctors - all in a single working day. Without a proper system, the staff end up spending more time searching for information than actually serving customers. A patient asks whether a particular antibiotic is available and the pharmacist has to flip through pages of a register to find out. An invoice gets calculated by hand and the arithmetic is off. A box of injections sits on the shelf a month past its expiry date because nobody checked.

The problem is not a lack of effort - it's a lack of the right tools. Our project addresses this directly. We designed the system around four core needs: knowing exactly what is in stock at all times, generating accurate bills without manual calculations, maintaining a searchable record of every patient and prescription, and receiving automatic alerts when something needs attention. Two background threads run continuously - one watching for medicines whose stock has dropped too low, and another checking expiry dates on a schedule - so the staff are warned before a problem becomes a crisis, not after.

Implementation Details

The project is written entirely in Java. For the visual side we chose JavaFX, which gave us proper desktop-grade UI components - tables, forms, dropdowns, and layout containers - without needing a web stack. The source code is spread across 32 files grouped under four packages. The model package is purely data - each class there describes one kind of record the system deals with. The service package contains all the processing logic, completely separated from any visual code. The ui package has one file per screen. The util package holds anything shared across the rest of the

code, like the file storage engine and the ID numbering system.

Keeping the service and UI layers separate was a deliberate choice we made after an early draft where a screen class was also doing database operations - it became unreadable fast. Once we separated them, each class had one clear job. The service classes don't know anything about buttons or text fields. The screens don't know anything about files or serialization. And because multiple threads run in the background accessing the same data, every method in the service layer that writes anything carries a lock to prevent two threads from colliding.

All 32 source files included in this project:

- Medicine.java
- MedicineCategory.java
- UserRole.java
- Customer.java
- Supplier.java
- Bill.java
- BillItem.java
- Prescription.java
- PrescriptionItem.java
- User.java
- InventoryManager.java
- BillingService.java
- CustomerService.java
- SupplierService.java
- PrescriptionService.java
- UserService.java
- ExpiryAlertThread.java
- StockAlertThread.java
- BillingThread.java
- BackupService.java
- PharmacyApp.java
- LoginScreen.java
- DashboardScreen.java
- InventoryScreen.java
- BillingScreen.java
- SupplierScreen.java
- CustomerScreen.java
- PrescriptionScreen.java
- ReportsScreen.java
- DataStore.java
- IDGenerator.java
- SampleDataLoader.java

Detailed Explanation of Each File

Model Package (com.pharmacy.model)

1. Medicine.java

This class describes a single medicine entry - everything worth knowing about one product sitting on the pharmacy shelf. The quantity field is typed as Integer rather than the primitive int, and price uses Double rather than double. This was intentional to show how Java automatically wraps and unwraps primitive values when assigning between the two forms. The class has two versions of the constructor: one that includes a supplier reference and one that leaves it out, demonstrating how constructors can be overloaded. A compareTo method was added so that when a list of medicines gets sorted, the natural ordering is alphabetical by name.

Code for Medicine.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Medicine model class representing a medicine in the pharmacy.
* Demonstrates: Encapsulation, Serialization, Constructors, Wrapper classes.
*/
public class Medicine implements Serializable, Comparable<Medicine> {
private static final long serialVersionUID = 1L;
private String medicineId;
private String name;
private String manufacturer;
private MedicineCategory category;
private String batchNumber;
private String expiryDate; // Format: YYYY-MM-DD
private Integer quantity; // Wrapper class demonstration
private Double price; // Wrapper class demonstration
private String supplierId;
// Default constructor
public Medicine() {
}
// Parameterized constructor
public Medicine(String medicineId, String name, String manufacturer,
MedicineCategory category, String batchNumber,
String expiryDate, int quantity, double price, String supplierId) {
this.medicineId = medicineId;
this.name = name;
this.manufacturer = manufacturer;
this.category = category;
this.batchNumber = batchNumber;
this.expiryDate = expiryDate;
this.quantity = quantity; // Autoboxing: int -> Integer
this.price = price; // Autoboxing: double -> Double
this.supplierId = supplierId;
}
// Constructor overloading (without supplier)
public Medicine(String medicineId, String name, String manufacturer,
MedicineCategory category, String batchNumber,
String expiryDate, int quantity, double price) {
this(medicineId, name, manufacturer, category, batchNumber,
expiryDate, quantity, price, "");
}
// Getters and Setters (Encapsulation)
public String getMedicineId() { return medicineId; }
public void setMedicineId(String medicineId) { this.medicineId = medicineId; }
```

```
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getManufacturer() { return manufacturer; }
public void setManufacturer(String manufacturer) { this.manufacturer = manufacturer; }
public MedicineCategory getCategory() { return category; }
public void setCategory(MedicineCategory category) { this.category = category; }
public String getBatchNumber() { return batchNumber; }
public void setBatchNumber(String batchNumber) { this.batchNumber = batchNumber; }
public String getExpiryDate() { return expiryDate; }
public void setExpiryDate(String expiryDate) { this.expiryDate = expiryDate; }
public Integer getQuantity() { return quantity; }
public void setQuantity(Integer quantity) { this.quantity = quantity; }
public Double getPrice() { return price; }
public void setPrice(Double price) { this.price = price; }
public String getSupplierId() { return supplierId; }
public void setSupplierId(String supplierId) { this.supplierId = supplierId; }
// Unboxing demonstration
public int getQuantityPrimitive() {
return quantity; // Unboxing: Integer -> int
}
public double getPricePrimitive() {
return price; // Unboxing: Double -> double
}
// Comparable implementation for sorting
@Override
public int compareTo(Medicine other) {
return this.name.compareToIgnoreCase(other.name);
}
@Override
public String toString() {
return medicineId + " | " + name + " | " + category + " | Qty: " + quantity
+ " | Rs." + String.format("%.2f", price) + " | Exp: " + expiryDate;
}
}
```

2. MedicineCategory.java

Rather than letting someone type "tablet" in one place and "Tablet" in another and have the system treat them as different things, we locked the allowed categories into an enum. The eight values cover the full range of what a typical pharmacy stocks: tablets, capsules, syrups, injections, ointments, drops, inhalers, and powders. Every category carries a display name alongside it so the UI can show something readable without any extra conversion step.

Code for MedicineCategory.java:

```
package com.pharmacy.model;
/**
* Enum representing different categories of medicines.
* Demonstrates: Enum usage as required by OSDL syllabus.
*/
public enum MedicineCategory {
TABLET("Tablet"),
CAPSULE("Capsule"),
SYRUP("Syrup"),
INJECTION("Injection"),
OINTMENT("Ointment"),
```

```
DROPS("Drops"),
INHALER("Inhaler"),
POWDER("Powder");
private final String displayName;
// Enum constructor
MedicineCategory(String displayName) {
this.displayName = displayName;
}
public String getDisplayName() {
return displayName;
}
@Override
public String toString() {
return displayName;
}
}
```

3. UserRole.java

Two roles exist in the system - one for administrators who need access to everything, and one for pharmacists whose access is more restricted. We put both in an enum rather than comparing strings at login time. Each role carries a boolean flag that answers whether that role is permitted to manage other user accounts. Checking permission anywhere in the app is then just a matter of calling that flag.

Code for UserRole.java:

```
package com.pharmacy.model;
/**
* Enum for role-based access control.
* Demonstrates: Enum with methods.
*/
public enum UserRole {
ADMIN("Administrator", true),
PHARMACIST("Pharmacist", false);
private final String displayName;
private final boolean canManageUsers;
UserRole(String displayName, boolean canManageUsers) {
this.displayName = displayName;
this.canManageUsers = canManageUsers;
}
public String getDisplayName() {
return displayName;
}
public boolean canManageUsers() {
return canManageUsers;
}
}
```

4. Customer.java

Five pieces of information describe a customer in this system: an ID we generate, their full name, phone number, email address, and physical address. The class is tagged so Java's built-in object serialisation can write it to disk and read it back on the next launch. The toString method produces a short summary string that shows up in dropdowns and log outputs.

Code for Customer.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Customer/Patient model class.
* Demonstrates: Encapsulation, Serialization.
*/
public class Customer implements Serializable {
private static final long serialVersionUID = 1L;
private String customerId;
private String name;
private String phone;
private String email;
private String address;
// Default constructor
public Customer() {
}
// Parameterized constructor
public Customer(String customerId, String name, String phone, String email, String address) {
this.customerId = customerId;
this.name = name;
this.phone = phone;
this.email = email;
this.address = address;
}
// Getters and Setters
public String getCustomerId() { return customerId; }
public void setCustomerId(String customerId) { this.customerId = customerId; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getAddress() { return address; }
public void setAddress(String address) { this.address = address; }
@Override
public String toString() {
return customerId + " | " + name + " | " + phone;
}
}
```

5. Supplier.java

A supplier record tracks the company that delivers medicines to the pharmacy. Beyond the company name we also store a specific contact person's name, their phone and email, and the company address. Each medicine in the inventory can carry a supplier ID pointing back to whichever supplier provided that batch. Like all the model classes, this one is serialisable so it persists across sessions.

Code for Supplier.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
```

```
* Supplier model class.
* Demonstrates: Encapsulation, Serialization.
*/
public class Supplier implements Serializable {
private static final long serialVersionUID = 1L;
private String supplierId;
private String companyName;
private String contactPerson;
private String phone;
private String email;
private String address;
// Default constructor
public Supplier() {
}
// Parameterized constructor
public Supplier(String supplierId, String companyName, String contactPerson,
String phone, String email, String address) {
this.supplierId = supplierId;
this.companyName = companyName;
this.contactPerson = contactPerson;
this.phone = phone;
this.email = email;
this.address = address;
}
// Getters and Setters
public String getSupplierId() { return supplierId; }
public void setSupplierId(String supplierId) { this.supplierId = supplierId; }
public String getCompanyName() { return companyName; }
public void setCompanyName(String companyName) { this.companyName = companyName; }
public String getContactPerson() { return contactPerson; }
public void setContactPerson(String contactPerson) { this.contactPerson = contactPerson; }
public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getAddress() { return address; }
public void setAddress(String address) { this.address = address; }
@Override
public String toString() {
return supplierId + " | " + companyName + " | " + contactPerson;
}
}
```

6. Bill.java

A bill is more than just a total - it records who bought what, when, and who at the pharmacy processed the sale. The list of individual items is stored inside the bill object itself. The discount logic was written as two separate methods sharing the same name: one accepts a percentage, the other accepts an amount and a flag that says whether to treat it as fixed or percentage. This is the clearest example of method overloading in the project.

Code for Bill.java:

```
package com.pharmacy.model;
import java.io.Serializable;
import java.util.ArrayList;
```

```
/**
* Bill/Invoice model class for sales transactions.
* Demonstrates: Encapsulation, Serialization, ArrayList, Wrapper classes.
*/
public class Bill implements Serializable {
private static final long serialVersionUID = 1L;
private String billId;
private String customerId;
private String customerName;
private String date;
private ArrayList<BillItem> items;
private Double totalAmount; // Wrapper class
private Double discount;
private Double netAmount;
private String billedBy; // username of the person who billed
// Default constructor
public Bill() {
this.items = new ArrayList<>();
this.totalAmount = 0.0;
this.discount = 0.0;
this.netAmount = 0.0;
}
// Parameterized constructor
public Bill(String billId, String customerId, String customerName,
String date, String billedBy) {
this.billId = billId;
this.customerId = customerId;
this.customerName = customerName;
this.date = date;
this.billedBy = billedBy;
this.items = new ArrayList<>();
this.totalAmount = 0.0;
this.discount = 0.0;
this.netAmount = 0.0;
}
// Add item and recalculate total
public void addItem(BillItem item) {
items.add(item);
recalculate();
}
// Recalculate totals
public void recalculate() {
double total = 0.0;
for (BillItem item : items) {
total += item.getSubtotal(); // Unboxing
}
this.totalAmount = total; // Autoboxing
this.netAmount = totalAmount - discount;
}
// Method overloading: apply discount by percentage
public void applyDiscount(double percentage) {
this.discount = totalAmount * (percentage / 100.0);
this.netAmount = totalAmount - discount;
}
// Method overloading: apply discount by fixed amount with a flag
public void applyDiscount(double amount, boolean isFixed) {
```

```
if (isFixed) {
this.discount = amount;
} else {
this.discount = totalAmount * (amount / 100.0);
}
this.netAmount = totalAmount - discount;
}
// Getters and Setters
public String getBillId() { return billId; }
public void setBillId(String billId) { this.billId = billId; }
public String getCustomerId() { return customerId; }
public void setCustomerId(String customerId) { this.customerId = customerId; }
public String getCustomerName() { return customerName; }
public void setCustomerName(String customerName) { this.customerName = customerName; }
public String getDate() { return date; }
public void setDate(String date) { this.date = date; }
public ArrayList<BillItem> getItems() { return items; }
public void setItems(ArrayList<BillItem> items) { this.items = items; }
public Double getTotalAmount() { return totalAmount; }
public void setTotalAmount(Double totalAmount) { this.totalAmount = totalAmount; }
public Double getDiscount() { return discount; }
public void setDiscount(Double discount) { this.discount = discount; }
public Double getNetAmount() { return netAmount; }
public void setNetAmount(Double netAmount) { this.netAmount = netAmount; }
public String getBilledBy() { return billedBy; }
public void setBilledBy(String billedBy) { this.billedBy = billedBy; }
@Override
public String toString() {
return billId + " | " + customerName + " | Rs." + String.format("%.2f", netAmount)
+ " | " + date;
}
}
```

7. BillItem.java

One row on a receipt maps to one BillItem object. It knows the medicine name, how many units were sold, the price per unit, and the line total. The numeric fields are all object types rather than primitives - Integer for the count and Double for the money values - so that autoboxing and unboxing are visibly happening when we assign plain numbers to them in the constructor.

Code for BillItem.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Individual item in a bill.
* Demonstrates: Encapsulation, Serialization, Wrapper classes.
*/
public class BillItem implements Serializable {
private static final long serialVersionUID = 1L;
private String medicineId;
private String medicineName;
private Integer quantity; // Wrapper class
private Double unitPrice; // Wrapper class
private Double subtotal; // Wrapper class
```

```
public BillItem() {
}
public BillItem(String medicineId, String medicineName, int quantity, double unitPrice) {
this.medicineId = medicineId;
this.medicineName = medicineName;
this.quantity = quantity; // Autoboxing
this.unitPrice = unitPrice; // Autoboxing
this.subtotal = quantity * unitPrice; // Autoboxing result
}
// Getters and Setters
public String getMedicineId() { return medicineId; }
public void setMedicineId(String medicineId) { this.medicineId = medicineId; }
public String getMedicineName() { return medicineName; }
public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
public Integer getQuantity() { return quantity; }
public void setQuantity(Integer quantity) { this.quantity = quantity; }
public Double getUnitPrice() { return unitPrice; }
public void setUnitPrice(Double unitPrice) { this.unitPrice = unitPrice; }
public Double getSubtotal() { return subtotal; }
public void setSubtotal(Double subtotal) { this.subtotal = subtotal; }
@Override
public String toString() {
return medicineName + " x" + quantity + " @ Rs." + String.format("%.2f", unitPrice)
+ " = Rs." + String.format("%.2f", subtotal);
}
}
```

8. Prescription.java

A doctor writes a prescription for a specific patient. This class captures that relationship: it stores which patient the prescription belongs to, the name of the prescribing doctor, the date, any special notes, and a collection of the individual medicines prescribed. The medicine list grows one entry at a time as items are added through the addItem method.

Code for Prescription.java:

```
package com.pharmacy.model;
import java.io.Serializable;
import java.util.ArrayList;
/**
* Prescription model class linking customer to prescribed medicines.
* Demonstrates: Encapsulation, Serialization, ArrayList usage.
*/
public class Prescription implements Serializable {
private static final long serialVersionUID = 1L;
private String prescriptionId;
private String customerId;
private String customerName;
private String doctorName;
private String date;
private ArrayList<PrescriptionItem> items;
private String notes;
// Default constructor
public Prescription() {
this.items = new ArrayList<>();
}
```

```
// Parameterized constructor
public Prescription(String prescriptionId, String customerId, String customerName,
String doctorName, String date, String notes) {
this.prescriptionId = prescriptionId;
this.customerId = customerId;
this.customerName = customerName;
this.doctorName = doctorName;
this.date = date;
this.notes = notes;
this.items = new ArrayList<>();
}
// Add item to prescription
public void addItem(PrescriptionItem item) {
items.add(item);
}
// Getters and Setters
public String getPrescriptionId() { return prescriptionId; }
public void setPrescriptionId(String prescriptionId) { this.prescriptionId = prescriptionId; }
public String getCustomerId() { return customerId; }
public void setCustomerId(String customerId) { this.customerId = customerId; }
public String getCustomerName() { return customerName; }
public void setCustomerName(String customerName) { this.customerName = customerName; }
public String getDoctorName() { return doctorName; }
public void setDoctorName(String doctorName) { this.doctorName = doctorName; }
public String getDate() { return date; }
public void setDate(String date) { this.date = date; }
public ArrayList<PrescriptionItem> getItems() { return items; }
public void setItems(ArrayList<PrescriptionItem> items) { this.items = items; }
public String getNotes() { return notes; }
public void setNotes(String notes) { this.notes = notes; }
@Override
public String toString() {
return prescriptionId + " | Patient: " + customerName + " | Dr. " + doctorName + " | " + date;
}
}
```

9. PrescriptionItem.java

Each medicine that appears in a prescription is stored as its own object. Beyond the medicine name and quantity, it also stores the dosage string - something like "1 tablet morning and night" - which is specific to that prescription and cannot come from the general inventory record.

Code for PrescriptionItem.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Individual item within a prescription.
* Demonstrates: Encapsulation, Serialization.
*/
public class PrescriptionItem implements Serializable {
private static final long serialVersionUID = 1L;
private String medicineId;
private String medicineName;
private Integer quantity;
private String dosage;
```

```
public PrescriptionItem() {
}
public PrescriptionItem(String medicineId, String medicineName, int quantity, String dosage) {
this.medicineId = medicineId;
this.medicineName = medicineName;
this.quantity = quantity; // Autoboxing
this.dosage = dosage;
}
public String getMedicineId() { return medicineId; }
public void setMedicineId(String medicineId) { this.medicineId = medicineId; }
public String getMedicineName() { return medicineName; }
public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
public Integer getQuantity() { return quantity; }
public void setQuantity(Integer quantity) { this.quantity = quantity; }
public String getDosage() { return dosage; }
public void setDosage(String dosage) { this.dosage = dosage; }
@Override
public String toString() {
return medicineName + " x" + quantity + " (" + dosage + ")";
}
}
```

10. User.java

Staff members who can log into the system are stored as User objects. The class holds a login name, a password, the person's full name, and their role. Password checking is handled by a method inside this class rather than in the login screen, so the credential comparison logic stays in one place. User objects are serialisable so accounts survive application restarts.

Code for User.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* User model for login and role-based access.
* Demonstrates: Encapsulation, Enum usage, Serialization.
*/
public class User implements Serializable {
private static final long serialVersionUID = 1L;
private String username;
private String password;
private String fullName;
private UserRole role;
// Default constructor
public User() {
}
// Parameterized constructor
public User(String username, String password, String fullName, UserRole role) {
this.username = username;
this.password = password;
this.fullName = fullName;
this.role = role;
}
// Validate login credentials
public boolean authenticate(String inputPassword) {
return this.password.equals(inputPassword);
```

```
}
// Getters and Setters
public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
public String getPassword() { return password; }
public void setPassword(String password) { this.password = password; }
public String getFullName() { return fullName; }
public void setFullName(String fullName) { this.fullName = fullName; }
public UserRole getRole() { return role; }
public void setRole(UserRole role) { this.role = role; }
@Override
public String toString() {
return username + " (" + role.getDisplayName() + ")";
}
}
```

Service Package (com.pharmacy.service)

11. InventoryManager.java

This service class is the backbone of the stock management side. It keeps the medicine data in two parallel structures at once: an ordered list for things like displaying all medicines in sequence, and a lookup table keyed by medicine ID for instant retrieval when you know what you're looking for. Because two background threads are also reading this data at runtime, every write operation is locked so only one thread can change anything at a time. Removing medicines while iterating is done through a traversal object that handles removal safely rather than modifying the list mid-loop. Sorting supports three modes: alphabetical by name using the natural ordering defined in Medicine.java, by price, and by expiry date.

Code for InventoryManager.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import com.pharmacy.model.MedicineCategory;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Iterator;
/**
* Manages the medicine inventory.
* Demonstrates: ArrayList, HashMap, Iterator, Collections.sort, Generics, Synchronization.
*/
public class InventoryManager {
private ArrayList<Medicine> medicineList;
private HashMap<String, Medicine> medicineMap; // Maps medicineId -> Medicine object
private DataStore<Medicine> dataStore;
private static final String DATA_FILE = "data/medicines.dat";
private static final String BACKUP_FILE = "data/backup_medicines.txt";
private static final int LOW_STOCK_THRESHOLD = 10;
public InventoryManager() {
this.dataStore = new DataStore<>(DATA_FILE);
this.medicineList = dataStore.loadAll();
this.medicineMap = new HashMap<>();
```

```
// Populate HashMap from loaded list
for (Medicine med : medicineList) {
medicineMap.put(med.getMedicineId(), med);
}
}
// Add medicine (synchronized for thread safety)
public synchronized void addMedicine(Medicine medicine) {
medicineList.add(medicine);
medicineMap.put(medicine.getMedicineId(), medicine);
saveData();
}
// Update medicine
public synchronized void updateMedicine(Medicine medicine) {
medicineMap.put(medicine.getMedicineId(), medicine);
// Update in list as well
for (int i = 0; i < medicineList.size(); i++) {
if (medicineList.get(i).getMedicineId().equals(medicine.getMedicineId())) {
medicineList.set(i, medicine);
break;
}
}
saveData();
}
// Delete medicine using Iterator
public synchronized void deleteMedicine(String medicineId) {
Iterator<Medicine> iterator = medicineList.iterator();
while (iterator.hasNext()) {
Medicine med = iterator.next();
if (med.getMedicineId().equals(medicineId)) {
iterator.remove();
break;
}
}
medicineMap.remove(medicineId);
saveData();
}
// Get medicine by ID using HashMap
public Medicine getMedicineById(String medicineId) {
return medicineMap.get(medicineId);
}
// Search medicines by name
public ArrayList<Medicine> searchByName(String name) {
ArrayList<Medicine> results = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getName().toLowerCase().contains(name.toLowerCase())) {
results.add(med);
}
}
return results;
}
// Filter by category
public ArrayList<Medicine> filterByCategory(MedicineCategory category) {
ArrayList<Medicine> results = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getCategory() == category) {
results.add(med);
```

```
}
}
return results;
}
// Sort by name using Collections.sort (Comparable)
public ArrayList<Medicine> getSortedByName() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted); // Uses Medicine's compareTo
return sorted;
}
// Sort by price using Comparator
public ArrayList<Medicine> getSortedByPrice() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted, new Comparator<Medicine>() {
@Override
public int compare(Medicine m1, Medicine m2) {
return Double.compare(m1.getPrice(), m2.getPrice());
}
});
return sorted;
}
// Sort by expiry date
public ArrayList<Medicine> getSortedByExpiry() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted, new Comparator<Medicine>() {
@Override
public int compare(Medicine m1, Medicine m2) {
return m1.getExpiryDate().compareTo(m2.getExpiryDate());
}
});
return sorted;
}
// Get low stock medicines
public ArrayList<Medicine> getLowStockMedicines() {
ArrayList<Medicine> lowStock = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getQuantity() < LOW_STOCK_THRESHOLD) {
lowStock.add(med);
}
}
return lowStock;
}
// Get expired medicines (compare with current date string YYYY-MM-DD)
public ArrayList<Medicine> getExpiredMedicines(String currentDate) {
ArrayList<Medicine> expired = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getExpiryDate().compareTo(currentDate) <= 0) {
expired.add(med);
}
}
return expired;
}
// Get medicines expiring soon (within 30 days approximation)
public ArrayList<Medicine> getExpiringSoon(String currentDate, String thresholdDate) {
ArrayList<Medicine> expiring = new ArrayList<>();
for (Medicine med : medicineList) {
```

```
if (med.getExpiryDate().compareTo(currentDate) > 0
&& med.getExpiryDate().compareTo(thresholdDate) <= 0) {
expiring.add(med);
}
}
return expiring;
}
// Reduce stock (for billing)
public synchronized boolean reduceStock(String medicineId, int quantity) {
Medicine med = medicineMap.get(medicineId);
if (med != null && med.getQuantity() >= quantity) {
med.setQuantity(med.getQuantity() - quantity);
saveData();
return true;
}
return false;
}
// Get all medicines
public ArrayList<Medicine> getAllMedicines() {
return medicineList;
}
// Save data
private void saveData() {
dataStore.saveAll(medicineList);
}
// Create backup
public void createBackup() {
dataStore.backupToText(medicineList, BACKUP_FILE);
}
public int getMedicineCount() {
return medicineList.size();
}
}
```

12. BillingService.java

This class sits between the billing screen and the data layer. When a bill comes in, it checks that sufficient stock exists for every item, deducts the quantities, saves the bill record, and writes a plain-text invoice file to disk. The invoice writing is done through a character stream writer pointed at a .txt file. Early in development this code used graphical border characters that only render correctly in UTF-8 but caused garbled output on Windows systems using a different default encoding, so those were replaced with plain dashes and pipes.

Code for BillingService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Bill;
import com.pharmacy.model.BillItem;
import com.pharmacy.util.DataStore;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
/**
* Manages billing and invoice operations.
* Demonstrates: ArrayList, Serialization, FileWriter, Method overloading.
*/
```

```
public class BillingService {
private ArrayList<Bill> billList;
private DataStore<Bill> dataStore;
private InventoryManager inventoryManager;
private static final String DATA_FILE = "data/bills.dat";
private static final String BACKUP_FILE = "data/backup_bills.txt";
private static final String INVOICE_DIR = "data/invoices/";
public BillingService(InventoryManager inventoryManager) {
this.inventoryManager = inventoryManager;
this.dataStore = new DataStore<>(DATA_FILE);
this.billList = dataStore.loadAll();
}
// Create and save a bill
public synchronized boolean createBill(Bill bill) {
// Reduce stock for each item
for (BillItem item : bill.getItems()) {
boolean success = inventoryManager.reduceStock(item.getMedicineId(), item.getQuantity());
if (!success) {
return false; // Insufficient stock
}
}
billList.add(bill);
saveData();
generateInvoice(bill);
return true;
}
/**
* Generate invoice as text file.
* Demonstrates: FileWriter usage.
*/
public void generateInvoice(Bill bill) {
java.io.File dir = new java.io.File(INVOICE_DIR);
if (!dir.exists()) {
dir.mkdirs();
}
String invoicePath = INVOICE_DIR + bill.getBillId() + "_invoice.txt";
try (FileWriter writer = new FileWriter(invoicePath)) {
writer.write("■■■
```

```
writer.write(String.format("%-37s %12.2f\n", "Total:", bill.getTotalAmount()));
writer.write(String.format("%-37s %12.2f\n", "Discount:", bill.getDiscount()));
writer.write(String.format("%-37s %12.2f\n", "Net Amount:", bill.getNetAmount()));
writer.write("■■■
```

Adding, updating, searching, and removing customer records all live here. The data sits in two structures simultaneously - a list that preserves insertion order and a map that gives instant access by customer ID. Removal uses a traversal cursor rather than index-based deletion, which avoids a structural modification error that Java throws if you delete from a collection while directly looping it. All changes are pushed to disk immediately through the generic storage class.

Code for CustomerService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Customer;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
/**
* Manages customer/patient records.
* Demonstrates: ArrayList, HashMap, Iterator, Generics.
*/
public class CustomerService {
private ArrayList<Customer> customerList;
private HashMap<String, Customer> customerMap;
private DataStore<Customer> dataStore;
private static final String DATA_FILE = "data/customers.dat";
private static final String BACKUP_FILE = "data/backup_customers.txt";
public CustomerService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.customerList = dataStore.loadAll();
this.customerMap = new HashMap<>();
for (Customer cust : customerList) {
customerMap.put(cust.getCustomerId(), cust);
}
}
public synchronized void addCustomer(Customer customer) {
customerList.add(customer);
customerMap.put(customer.getCustomerId(), customer);
saveData();
}
public synchronized void updateCustomer(Customer customer) {
customerMap.put(customer.getCustomerId(), customer);
for (int i = 0; i < customerList.size(); i++) {
if (customerList.get(i).getCustomerId().equals(customer.getCustomerId())) {
customerList.set(i, customer);
break;
}
}
saveData();
}
public synchronized void deleteCustomer(String customerId) {
Iterator<Customer> iterator = customerList.iterator();
while (iterator.hasNext()) {
Customer cust = iterator.next();
if (cust.getCustomerId().equals(customerId)) {
iterator.remove();
break;
}
}
```

```
customerMap.remove(customerId);
saveData();
}
public Customer getCustomerById(String customerId) {
return customerMap.get(customerId);
}
public ArrayList<Customer> searchByName(String name) {
ArrayList<Customer> results = new ArrayList<>();
for (Customer cust : customerList) {
if (cust.getName().toLowerCase().contains(name.toLowerCase())) {
results.add(cust);
}
}
return results;
}
public ArrayList<Customer> getAllCustomers() {
return customerList;
}
private void saveData() {
dataStore.saveAll(customerList);
}
public void createBackup() {
dataStore.backupToText(customerList, BACKUP_FILE);
}
public int getCustomerCount() {
return customerList.size();
}
}
```

14. SupplierService.java

Structured identically to the customer service but operating on supplier records instead. The same dual-storage pattern applies: list plus map. The search feature here matches against company name rather than a person's name, using a case-folded substring comparison so partial matches work regardless of how the user types the name.

Code for SupplierService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Supplier;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
/**
* Manages supplier records.
* Demonstrates: ArrayList, HashMap, Iterator, Generics.
*/
public class SupplierService {
private ArrayList<Supplier> supplierList;
private HashMap<String, Supplier> supplierMap;
private DataStore<Supplier> dataStore;
private static final String DATA_FILE = "data/suppliers.dat";
private static final String BACKUP_FILE = "data/backup_suppliers.txt";
public SupplierService() {
this.dataStore = new DataStore<>(DATA_FILE);
```

```
this.supplierList = dataStore.loadAll();
this.supplierMap = new HashMap<>();
for (Supplier sup : supplierList) {
supplierMap.put(sup.getSupplierId(), sup);
}
}
public synchronized void addSupplier(Supplier supplier) {
supplierList.add(supplier);
supplierMap.put(supplier.getSupplierId(), supplier);
saveData();
}
public synchronized void updateSupplier(Supplier supplier) {
supplierMap.put(supplier.getSupplierId(), supplier);
for (int i = 0; i < supplierList.size(); i++) {
if (supplierList.get(i).getSupplierId().equals(supplier.getSupplierId())) {
supplierList.set(i, supplier);
break;
}
}
saveData();
}
public synchronized void deleteSupplier(String supplierId) {
Iterator<Supplier> iterator = supplierList.iterator();
while (iterator.hasNext()) {
Supplier sup = iterator.next();
if (sup.getSupplierId().equals(supplierId)) {
iterator.remove();
break;
}
}
supplierMap.remove(supplierId);
saveData();
}
public Supplier getSupplierById(String supplierId) {
return supplierMap.get(supplierId);
}
public ArrayList<Supplier> searchByName(String name) {
ArrayList<Supplier> results = new ArrayList<>();
for (Supplier sup : supplierList) {
if (sup.getCompanyName().toLowerCase().contains(name.toLowerCase())) {
results.add(sup);
}
}
return results;
}
public ArrayList<Supplier> getAllSuppliers() {
return supplierList;
}
private void saveData() {
dataStore.saveAll(supplierList);
}
public void createBackup() {
dataStore.backupToText(supplierList, BACKUP_FILE);
}
public int getSupplierCount() {
```

```
return supplierList.size();
}
}
```

15. PrescriptionService.java

Prescription records are added, retrieved, and deleted through this class. The lookup-by-patient method is particularly useful in the UI - when a staff member pulls up a customer's profile, all their past prescriptions can be fetched with a single call. Storage and retrieval from disk is entirely delegated to the generic DataStore class.

Code for PrescriptionService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Prescription;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.HashMap;
/**
* Manages prescription records.
* Demonstrates: ArrayList, HashMap, Generics, Serialization.
*/
public class PrescriptionService {
private ArrayList<Prescription> prescriptionList;
private HashMap<String, Prescription> prescriptionMap;
private DataStore<Prescription> dataStore;
private static final String DATA_FILE = "data/prescriptions.dat";
private static final String BACKUP_FILE = "data/backup_prescriptions.txt";
public PrescriptionService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.prescriptionList = dataStore.loadAll();
this.prescriptionMap = new HashMap<>();
for (Prescription pres : prescriptionList) {
prescriptionMap.put(pres.getPrescriptionId(), pres);
}
}
public synchronized void addPrescription(Prescription prescription) {
prescriptionList.add(prescription);
prescriptionMap.put(prescription.getPrescriptionId(), prescription);
saveData();
}
public synchronized void deletePrescription(String prescriptionId) {
prescriptionList.removeIf(p -> p.getPrescriptionId().equals(prescriptionId));
prescriptionMap.remove(prescriptionId);
saveData();
}
public Prescription getPrescriptionById(String prescriptionId) {
return prescriptionMap.get(prescriptionId);
}
public ArrayList<Prescription> getByCustomerId(String customerId) {
ArrayList<Prescription> results = new ArrayList<>();
for (Prescription pres : prescriptionList) {
if (pres.getCustomerId().equals(customerId)) {
results.add(pres);
}
}
```

```
return results;
}
public ArrayList<Prescription> getAllPrescriptions() {
return prescriptionList;
}
private void saveData() {
dataStore.saveAll(prescriptionList);
}
public void createBackup() {
dataStore.backupToText(prescriptionList, BACKUP_FILE);
}
public int getPrescriptionCount() {
return prescriptionList.size();
}
}
```

16. UserService.java

The first time the application launches with no saved user data, this class quietly creates two default accounts so there is always something to log in with. When a login attempt comes in, it walks the user list looking for a name match and then asks the User object itself to verify the password - so the password comparison logic is not spread across multiple files.

Code for UserService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.User;
import com.pharmacy.model.UserRole;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
/**
* Manages user authentication and accounts.
* Demonstrates: ArrayList, Serialization, role-based access.
*/
public class UserService {
private ArrayList<User> userList;
private DataStore<User> dataStore;
private User currentUser;
private static final String DATA_FILE = "data/users.dat";
public UserService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.userList = dataStore.loadAll();
// Create default admin if no users exist
if (userList.isEmpty()) {
createDefaultUsers();
}
}
// Create default users for first run
private void createDefaultUsers() {
userList.add(new User("admin", "admin123", "System Administrator", UserRole.ADMIN));
userList.add(new User("pharmacist", "pharma123", "Default Pharmacist", UserRole.PHARMACIST));
dataStore.saveAll(userList);
}
// Authenticate user
public User authenticate(String username, String password) {
for (User user : userList) {
```

```
if (user.getUsername().equals(username) && user.authenticate(password)) {
this.currentUser = user;
return user;
}
}
return null;
}
public User getCurrentUser() {
return currentUser;
}
public void setCurrentUser(User user) {
this.currentUser = user;
}
public void addUser(User user) {
userList.add(user);
dataStore.saveAll(userList);
}
public ArrayList<User> getAllUsers() {
return userList;
}
public void logout() {
this.currentUser = null;
}
}
```

17. ExpiryAlertThread.java

This is one of two background workers in the system and it uses direct class extension rather than the interface approach. It is flagged as a daemon so the JVM does not wait for it to finish before shutting down. After pausing for thirty seconds it wakes up and asks the inventory manager for two lists: medicines already past their date, and medicines within thirty days of expiring. A wait/notify pair lets other threads block on this one and receive the results safely once they are ready.

Code for ExpiryAlertThread.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
/**
* Background thread that checks for expired and soon-to-expire medicines.
* Demonstrates: Thread class, sleep(), join(), synchronized, Runnable.
*/
public class ExpiryAlertThread extends Thread {
private InventoryManager inventoryManager;
private volatile boolean running = true;
private ArrayList<Medicine> expiredMedicines;
private ArrayList<Medicine> expiringMedicines;
private final Object lock = new Object();
private AlertListener listener;
// Interface for callback (Abstraction demonstration)
public interface AlertListener {
void onAlertUpdate(ArrayList<Medicine> expired, ArrayList<Medicine> expiringSoon);
}
```

```
public ExpiryAlertThread(InventoryManager inventoryManager) {
super("ExpiryAlertThread");
this.inventoryManager = inventoryManager;
this.expiredMedicines = new ArrayList<>();
this.expiringMedicines = new ArrayList<>();
setDaemon(true); // Daemon thread - will stop when main thread stops
}
public void setAlertListener(AlertListener listener) {
this.listener = listener;
}
@Override
public void run() {
while (running) {
try {
checkExpiry();
// Sleep for 30 seconds between checks
Thread.sleep(30000);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}
// Synchronized method for thread-safe expiry checking
private synchronized void checkExpiry() {
SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
String currentDate = sdf.format(new Date());
// Calculate date 30 days from now
Calendar cal = Calendar.getInstance();
cal.add(Calendar.DAY_OF_MONTH, 30);
String thresholdDate = sdf.format(cal.getTime());
synchronized (lock) {
expiredMedicines = inventoryManager.getExpiredMedicines(currentDate);
expiringMedicines = inventoryManager.getExpiringSoon(currentDate, thresholdDate);
// Notify waiting threads using wait/notify mechanism
lock.notifyAll();
}
// Callback to listener
if (listener != null) {
listener.onAlertUpdate(expiredMedicines, expiringMedicines);
}
}
// Method for other threads to wait for alert data
public ArrayList<Medicine> waitForExpiredList() throws InterruptedException {
synchronized (lock) {
lock.wait(5000); // Wait up to 5 seconds
return new ArrayList<>(expiredMedicines);
}
}
public ArrayList<Medicine> getExpiredMedicines() {
synchronized (lock) {
return new ArrayList<>(expiredMedicines);
}
}
public ArrayList<Medicine> getExpiringMedicines() {
```

```
synchronized (lock) {
return new ArrayList<>(expiringMedicines);
}
}
public void stopChecking() {
running = false;
this.interrupt();
}
}
```

18. StockAlertThread.java

The second background worker, written using the interface-based threading approach rather than subclassing. A volatile flag controls whether the loop keeps running - marking it volatile ensures the main thread's update to that flag is visible to this thread immediately without caching issues. The stock check itself is wrapped in a lock so that if both this thread and the inventory screen try to read inventory data at the same moment, they queue rather than collide.

Code for StockAlertThread.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import java.util.ArrayList;
/**
* Background thread that monitors low stock levels.
* Demonstrates: Runnable interface, Thread, sleep(), synchronized.
*/
public class StockAlertThread implements Runnable {
private InventoryManager inventoryManager;
private volatile boolean running = true;
private ArrayList<Medicine> lowStockMedicines;
private AlertListener listener;
// Callback interface (Abstraction)
public interface AlertListener {
void onLowStockUpdate(ArrayList<Medicine> lowStockMedicines);
}
public StockAlertThread(InventoryManager inventoryManager) {
this.inventoryManager = inventoryManager;
this.lowStockMedicines = new ArrayList<>();
}
public void setAlertListener(AlertListener listener) {
this.listener = listener;
}
@Override
public void run() {
while (running) {
try {
checkStock();
Thread.sleep(30000); // Check every 30 seconds
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}
private synchronized void checkStock() {
```

```
lowStockMedicines = inventoryManager.getLowStockMedicines();
if (listener != null) {
listener.onLowStockUpdate(lowStockMedicines);
}
}
public synchronized ArrayList<Medicine> getLowStockMedicines() {
return new ArrayList<>(lowStockMedicines);
}
public void stopChecking() {
running = false;
}
}
```

19. BillingThread.java

Processing a bill - checking stock, writing the invoice file, saving the record - takes a noticeable amount of time if done on the same thread as the UI. Doing so would freeze the interface while the user waits. This class moves that work onto a separate thread. When finished, it needs to update labels on screen, but UI elements can only be touched from the main thread, so the result is handed back through a scheduled task that runs on the correct thread. A join call in a separate watcher thread demonstrates how one thread can pause until another has fully completed.

Code for BillingThread.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Bill;
/**
* Multi-threaded billing simulation.
* Demonstrates: Thread, Runnable, synchronized, sleep(), join().
*
* Simulates billing processing in a separate thread to keep UI responsive.
*/
public class BillingThread extends Thread {
private BillingService billingService;
private Bill bill;
private boolean success;
private String message;
private BillingCallback callback;
// Callback interface for billing result (Abstraction)
public interface BillingCallback {
void onBillingComplete(boolean success, String message, Bill bill);
}
public BillingThread(BillingService billingService, Bill bill, BillingCallback callback) {
super("BillingThread-" + bill.getBillId());
this.billingService = billingService;
this.bill = bill;
this.callback = callback;
this.success = false;
this.message = "";
}
@Override
public void run() {
try {
// Simulate processing time
System.out.println("[" + getName() + "] Processing bill " + bill.getBillId() + "...");
Thread.sleep(1000); // Simulate processing delay
```

```
// Process the bill
synchronized (billingService) {
success = billingService.createBill(bill);
}
if (success) {
message = "Bill " + bill.getBillId() + " processed successfully!";
System.out.println("[" + getName() + "] " + message);
} else {
message = "Failed to process bill " + bill.getBillId() + ". Insufficient stock.";
System.out.println("[" + getName() + "] " + message);
}
// Simulate invoice generation time
Thread.sleep(500);
} catch (InterruptedException e) {
message = "Billing interrupted for " + bill.getBillId();
Thread.currentThread().interrupt();
}
// Callback
if (callback != null) {
callback.onBillingComplete(success, message, bill);
}
}
public boolean isSuccess() {
return success;
}
public String getMessage() {
return message;
}
}
```

20. BackupService.java

Written as a task rather than a thread subclass, this class can be handed to any thread the caller creates. When it runs, it goes through each of the five service classes one by one and asks each to write a plain-text copy of its data to the backup folder, with brief pauses between each one. A callback interface lets the caller know when everything is done so the UI can show a confirmation.

Code for BackupService.java:

```
package com.pharmacy.service;
/**
* Backup service that creates file backups of all data.
* Demonstrates: Runnable, Thread, FileWriter (via DataStore).
*/
public class BackupService implements Runnable {
private InventoryManager inventoryManager;
private CustomerService customerService;
private SupplierService supplierService;
private BillingService billingService;
private PrescriptionService prescriptionService;
private BackupCallback callback;
public interface BackupCallback {
void onBackupComplete(String message);
}
public BackupService(InventoryManager inventoryManager, CustomerService customerService,
SupplierService supplierService, BillingService billingService,
```

```
PrescriptionService prescriptionService) {
this.inventoryManager = inventoryManager;
this.customerService = customerService;
this.supplierService = supplierService;
this.billingService = billingService;
this.prescriptionService = prescriptionService;
}
public void setCallback(BackupCallback callback) {
this.callback = callback;
}
@Override
public void run() {
try {
System.out.println("[BackupService] Starting backup...");
Thread.sleep(500);
inventoryManager.createBackup();
Thread.sleep(200);
customerService.createBackup();
Thread.sleep(200);
supplierService.createBackup();
Thread.sleep(200);
billingService.createBackup();
Thread.sleep(200);
prescriptionService.createBackup();
Thread.sleep(200);
String msg = "All data backed up successfully!";
System.out.println("[BackupService] " + msg);
if (callback != null) {
callback.onBackupComplete(msg);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
if (callback != null) {
callback.onBackupComplete("Backup interrupted!");
}
}
}
}
```

UI Package (com.pharmacy.ui)

21. PharmacyApp.java

The application's starting point. It extends the JavaFX base class and overrides the startup method where all initialisation happens: service objects are created, the two monitoring workers are started, demo data is loaded if the data folder is empty, and the login window appears. Screen navigation works by calling one of several named methods, each of which builds the target screen and hands it to the window to display.

Code for PharmacyApp.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import com.pharmacy.service.*;
import com.pharmacy.util.SampleDataLoader;
```

```
import javafx.application.Application;
import javafx.application.Platform;
import javafx.stage.Stage;
import java.io.File;
/**
* Main JavaFX Application class - Entry point for the Pharmacy Management System.
* Demonstrates: JavaFX Application, Stage management, Inheritance (extends Application).
*/
public class PharmacyApp extends Application {
// Service instances (shared across all screens)
private UserService userService;
private InventoryManager inventoryManager;
private CustomerService customerService;
private SupplierService supplierService;
private BillingService billingService;
private PrescriptionService prescriptionService;
// Alert threads
private ExpiryAlertThread expiryAlertThread;
private Thread stockAlertThreadHandle;
private StockAlertThread stockAlertRunnable;
private Stage primaryStage;
@Override
public void start(Stage primaryStage) {
this.primaryStage = primaryStage;
// Create data directory
new File("data/invoices").mkdirs();
// Initialize all services
userService = new UserService();
inventoryManager = new InventoryManager();
customerService = new CustomerService();
supplierService = new SupplierService();
billingService = new BillingService(inventoryManager);
prescriptionService = new PrescriptionService();
// Load sample data if empty
SampleDataLoader.loadSampleData(inventoryManager, customerService, supplierService);
// Start background alert threads (Multithreading demonstration)
startAlertThreads();
// Show login screen
showLoginScreen();
primaryStage.setTitle("PharmaCare - Pharmacy Management System");
primaryStage.setWidth(1100);
primaryStage.setHeight(700);
primaryStage.setOnCloseRequest(e -> {
stopAlertThreads();
Platform.exit();
System.exit(0);
});
primaryStage.show();
}
/**
* Start background threads for monitoring.
* Demonstrates: Thread creation (both Thread subclass and Runnable), start(), daemon threads.
*/
private void startAlertThreads() {
// Thread extending Thread class
```

```
expiryAlertThread = new ExpiryAlertThread(inventoryManager);
expiryAlertThread.start();
// Thread using Runnable interface
stockAlertRunnable = new StockAlertThread(inventoryManager);
stockAlertThreadHandle = new Thread(stockAlertRunnable, "StockAlertThread");
stockAlertThreadHandle.setDaemon(true);
stockAlertThreadHandle.start();
}
private void stopAlertThreads() {
if (expiryAlertThread != null) {
expiryAlertThread.stopChecking();
}
if (stockAlertRunnable != null) {
stockAlertRunnable.stopChecking();
}
}
// Show login screen
public void showLoginScreen() {
LoginScreen loginScreen = new LoginScreen(this, userService);
primaryStage.setScene(loginScreen.getScene());
}
// Show dashboard after login
public void showDashboard(User user) {
DashboardScreen dashboard = new DashboardScreen(this, user);
primaryStage.setScene(dashboard.getScene());
}
// Show inventory module
public void showInventoryScreen() {
InventoryScreen screen = new InventoryScreen(this, inventoryManager, supplierService);
primaryStage.setScene(screen.getScene());
}
// Show billing module
public void showBillingScreen() {
BillingScreen screen = new BillingScreen(this, billingService, inventoryManager,
customerService, userService);
primaryStage.setScene(screen.getScene());
}
// Show supplier module
public void showSupplierScreen() {
SupplierScreen screen = new SupplierScreen(this, supplierService);
primaryStage.setScene(screen.getScene());
}
// Show customer module
public void showCustomerScreen() {
CustomerScreen screen = new CustomerScreen(this, customerService);
primaryStage.setScene(screen.getScene());
}
// Show prescription module
public void showPrescriptionScreen() {
PrescriptionScreen screen = new PrescriptionScreen(this, prescriptionService,
customerService, inventoryManager);
primaryStage.setScene(screen.getScene());
}
// Show reports module
public void showReportsScreen() {
```

```
ReportsScreen screen = new ReportsScreen(this, inventoryManager, billingService,
customerService, supplierService,
expiryAlertThread, stockAlertRunnable);
primaryStage.setScene(screen.getScene());
}
// Getters for services
public UserService getUserService() { return userService; }
public InventoryManager getInventoryManager() { return inventoryManager; }
public CustomerService getCustomerService() { return customerService; }
public SupplierService getSupplierService() { return supplierService; }
public BillingService getBillingService() { return billingService; }
public PrescriptionService getPrescriptionService() { return prescriptionService; }
public ExpiryAlertThread getExpiryAlertThread() { return expiryAlertThread; }
public StockAlertThread getStockAlertRunnable() { return stockAlertRunnable; }
public Stage getPrimaryStage() { return primaryStage; }
/**
* Main method - entry point.
*/
public static void main(String[] args) {
launch(args);
}
}
```

22. LoginScreen.java

A grid-based form with username and password fields, the second of which masks its input. On submission, the credentials go to the user service for checking. A wrong entry makes a red message appear below the form. A correct entry stores the authenticated user and hands control to the dashboard screen. The role attached to the user object at this point determines what options will appear throughout the rest of the session.

Code for LoginScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import com.pharmacy.service.UserService;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Login Screen with role-based authentication.
* Demonstrates: JavaFX (GridPane, TextField, Button, Label), Event handling.
*/
public class LoginScreen {
private Scene scene;
private PharmacyApp app;
private UserService userService;
public LoginScreen(PharmacyApp app, UserService userService) {
this.app = app;
this.userService = userService;
createScene();
}
```

```
private void createScene() {
// Main layout
VBox mainLayout = new VBox(20);
mainLayout.setAlignment(Pos.CENTER);
mainLayout.setPadding(new Insets(50));
mainLayout.setStyle("-fx-background-color: #2c3e50;");
// Title
Label titleLabel = new Label("PharmaCare");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 36));
titleLabel.setStyle("-fx-text-fill: #ecf0f1;");
Label subtitleLabel = new Label("Pharmacy Management System");
subtitleLabel.setFont(Font.font("Arial", FontWeight.NORMAL, 16));
subtitleLabel.setStyle("-fx-text-fill: #bdc3c7;");
// Login form using GridPane
GridPane grid = new GridPane();
grid.setAlignment(Pos.CENTER);
grid.setHgap(10);
grid.setVgap(15);
grid.setPadding(new Insets(30));
grid.setMaxWidth(400);
grid.setStyle("-fx-background-color: #34495e; -fx-background-radius: 10;");
Label loginTitle = new Label("Login");
loginTitle.setFont(Font.font("Arial", FontWeight.BOLD, 20));
loginTitle.setStyle("-fx-text-fill: #ecf0f1;");
grid.add(loginTitle, 0, 0, 2, 1);
Label userLabel = new Label("Username:");
userLabel.setStyle("-fx-text-fill: #ecf0f1;");
TextField usernameField = new TextField();
usernameField.setPromptText("Enter username");
usernameField.setPrefWidth(250);
grid.add(userLabel, 0, 1);
grid.add(usernameField, 1, 1);
Label passLabel = new Label("Password:");
passLabel.setStyle("-fx-text-fill: #ecf0f1;");
PasswordField passwordField = new PasswordField();
passwordField.setPromptText("Enter password");
grid.add(passLabel, 0, 2);
grid.add(passwordField, 1, 2);
Label messageLabel = new Label();
messageLabel.setStyle("-fx-text-fill: #e74c3c;");
Button loginBtn = new Button("Login");
loginBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; " +
"-fx-font-size: 14; -fx-pref-width: 250; -fx-cursor: hand;");
loginBtn.setOnAction(e -> {
String username = usernameField.getText().trim();
String password = passwordField.getText().trim();
if (username.isEmpty() || password.isEmpty()) {
messageLabel.setText("Please enter both username and password.");
return;
}
User user = userService.authenticate(username, password);
if (user != null) {
app.showDashboard(user);
} else {
messageLabel.setText("Invalid credentials! Please try again.");
```

```
passwordField.clear();
}
});
// Allow Enter key to login
passwordField.setOnAction(e -> loginBtn.fire());
usernameField.setOnAction(e -> loginBtn.fire());
grid.add(loginBtn, 0, 3, 2, 1);
grid.add(messageLabel, 0, 4, 2, 1);
// Default credentials hint
Label hintLabel = new Label("Default: admin/admin123 or pharmacist/pharma123");
hintLabel.setStyle("-fx-text-fill: #7f8c8d; -fx-font-size: 11;");
mainLayout.getChildren().addAll(titleLabel, subtitleLabel, grid, hintLabel);
scene = new Scene(mainLayout, 1100, 700);
}
public Scene getScene() {
return scene;
}
}
```

23. DashboardScreen.java

The home screen after login is deliberately simple - a dark header bar showing who is logged in, and six large buttons arranged in a grid, one for each module. A logout button in the header clears the session and returns to the login screen. The whole layout uses a border container at the outer level with a horizontal bar at the top and a grid in the centre.

Code for DashboardScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Dashboard screen - main navigation hub after login.
* Demonstrates: JavaFX (VBox, HBox, Button, Label, GridPane), Event handling.
*/
public class DashboardScreen {
private Scene scene;
private PharmacyApp app;
private User currentUser;
public DashboardScreen(PharmacyApp app, User currentUser) {
this.app = app;
this.currentUser = currentUser;
createScene();
}
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(20);
topBar.setPadding(new Insets(15, 25, 15, 25));
```

```
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Label titleLabel = new Label("PharmaCare Dashboard");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 22));
titleLabel.setStyle("-fx-text-fill: white;");
Region spacer = new Region();
HBox.setHgrow(spacer, Priority.ALWAYS);
Label userLabel = new Label("Welcome, " + currentUser.getFullName()
+ " (" + currentUser.getRole().getDisplayName() + ")");
userLabel.setStyle("-fx-text-fill: #bdc3c7; -fx-font-size: 13;");
Button logoutBtn = new Button("Logout");
logoutBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
logoutBtn.setOnAction(e -> {
app.getUserService().logout();
app.showLoginScreen();
});
topBar.getChildren().addAll(titleLabel, spacer, userLabel, logoutBtn);
root.setTop(topBar);
// Center - Module buttons in a grid
GridPane moduleGrid = new GridPane();
moduleGrid.setAlignment(Pos.CENTER);
moduleGrid.setHgap(25);
moduleGrid.setVgap(25);
moduleGrid.setPadding(new Insets(40));
// Module buttons
Button inventoryBtn = createModuleButton("Inventory\nManagement",
"Manage medicines, stock,\nbatch & expiry tracking", "#3498db");
inventoryBtn.setOnAction(e -> app.showInventoryScreen());
Button billingBtn = createModuleButton("Sales &\nBilling",
"Create bills, generate\ninvoices for customers", "#27ae60");
billingBtn.setOnAction(e -> app.showBillingScreen());
Button supplierBtn = createModuleButton("Supplier\nManagement",
"Manage supplier records\nand contact details", "#e67e22");
supplierBtn.setOnAction(e -> app.showSupplierScreen());
Button customerBtn = createModuleButton("Customer\nRecords",
"Manage patient/customer\ninformation", "#9b59b6");
customerBtn.setOnAction(e -> app.showCustomerScreen());
Button prescriptionBtn = createModuleButton("Prescription\nHandling",
"Record and manage\nprescription details", "#1abc9c");
prescriptionBtn.setOnAction(e -> app.showPrescriptionScreen());
Button reportsBtn = createModuleButton("Reports &\nAlerts",
"View sales reports, stock\n& expiry alerts", "#e74c3c");
reportsBtn.setOnAction(e -> app.showReportsScreen());
// Arrange in 3x2 grid
moduleGrid.add(inventoryBtn, 0, 0);
moduleGrid.add(billingBtn, 1, 0);
moduleGrid.add(supplierBtn, 2, 0);
moduleGrid.add(customerBtn, 0, 1);
moduleGrid.add(prescriptionBtn, 1, 1);
moduleGrid.add(reportsBtn, 2, 1);
root.setCenter(moduleGrid);
// Bottom status bar
HBox statusBar = new HBox(20);
statusBar.setPadding(new Insets(10, 25, 10, 25));
```

```
statusBar.setStyle("-fx-background-color: #34495e;");
Label statusLabel = new Label("Medicines: " + app.getInventoryManager().getMedicineCount()
+ " | Customers: " + app.getCustomerService().getCustomerCount()
+ " | Suppliers: " + app.getSupplierService().getSupplierCount()
+ " | Bills: " + app.getBillingService().getBillCount());
statusLabel.setStyle("-fx-text-fill: #bdc3c7; -fx-font-size: 12;");
statusBar.getChildren().add(statusLabel);
root.setBottom(statusBar);
scene = new Scene(root, 1100, 700);
}
// Helper to create styled module buttons
private Button createModuleButton(String title, String description, String color) {
VBox content = new VBox(8);
content.setAlignment(Pos.CENTER);
Label titleLbl = new Label(title);
titleLbl.setFont(Font.font("Arial", FontWeight.BOLD, 16));
titleLbl.setStyle("-fx-text-fill: white; -fx-text-alignment: center;");
Label descLbl = new Label(description);
descLbl.setStyle("-fx-text-fill: #ecf0f1; -fx-font-size: 11; -fx-text-alignment: center;");
content.getChildren().addAll(titleLbl, descLbl);
Button btn = new Button();
btn.setGraphic(content);
btn.setPrefSize(230, 130);
btn.setStyle("-fx-background-color: " + color + "; -fx-background-radius: 10; -fx-cursor:
hand;");
return btn;
}
public Scene getScene() {
return scene;
}
}
```

24. InventoryScreen.java

This screen has the most going on visually. A scrollable table fills most of the space, showing every medicine in stock. A form on the left handles adding new entries and editing existing ones - clicking any table row copies that row's data into the form automatically. Above the table, a text search box, a sort selector, and a category filter all operate on the displayed list independently of each other.

Code for InventoryScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Medicine;
import com.pharmacy.model.MedicineCategory;
import com.pharmacy.service.InventoryManager;
import com.pharmacy.service.SupplierService;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
```

```
import javafx.scene.text.FontWeight;
import java.util.ArrayList;
/**
* Inventory Management Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Buttons, ComboBox.
*/
public class InventoryScreen {
private Scene scene;
private PharmacyApp app;
private InventoryManager inventoryManager;
private SupplierService supplierService;
private TableView<Medicine> tableView;
private ObservableList<Medicine> tableData;
public InventoryScreen(PharmacyApp app, InventoryManager inventoryManager,
SupplierService supplierService) {
this.app = app;
this.inventoryManager = inventoryManager;
this.supplierService = supplierService;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = createTopBar("Inventory Management");
root.setTop(topBar);
// Left side - Form
VBox formBox = createForm();
root.setLeft(formBox);
// Center - Table
VBox tableBox = createTable();
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private HBox createTopBar(String title) {
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label(title);
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
return topBar;
}
private VBox createForm() {
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
```

```
Label formTitle = new Label("Add / Edit Medicine");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
// Form fields using GridPane
GridPane grid = new GridPane();
grid.setVgap(8);
grid.setHgap(8);
TextField idField = new TextField();
idField.setEditable(false);
idField.setPromptText("Auto-generated");
TextField nameField = new TextField();
nameField.setPromptText("Medicine name");
TextField mfgField = new TextField();
mfgField.setPromptText("Manufacturer");
ComboBox<MedicineCategory> categoryBox = new ComboBox<>();
categoryBox.getItems().addAll(MedicineCategory.values());
categoryBox.setPromptText("Select Category");
categoryBox.setPrefWidth(180);
TextField batchField = new TextField();
batchField.setPromptText("Batch number");
TextField expiryField = new TextField();
expiryField.setPromptText("YYYY-MM-DD");
TextField qtyField = new TextField();
qtyField.setPromptText("Quantity");
TextField priceField = new TextField();
priceField.setPromptText("Price (Rs.)");
TextField supplierField = new TextField();
supplierField.setPromptText("Supplier ID");
int row = 0;
grid.add(new Label("ID:"), 0, row); grid.add(idField, 1, row++);
grid.add(new Label("Name:"), 0, row); grid.add(nameField, 1, row++);
grid.add(new Label("Manufacturer:"), 0, row); grid.add(mfgField, 1, row++);
grid.add(new Label("Category:"), 0, row); grid.add(categoryBox, 1, row++);
grid.add(new Label("Batch No:"), 0, row); grid.add(batchField, 1, row++);
grid.add(new Label("Expiry Date:"), 0, row); grid.add(expiryField, 1, row++);
grid.add(new Label("Quantity:"), 0, row); grid.add(qtyField, 1, row++);
grid.add(new Label("Price:"), 0, row); grid.add(priceField, 1, row++);
grid.add(new Label("Supplier ID:"), 0, row); grid.add(supplierField, 1, row++);
Label statusLabel = new Label();
statusLabel.setStyle("-fx-text-fill: #27ae60;");
// Buttons
HBox btnBox = new HBox(8);
Button addBtn = new Button("Add");
addBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-cursor: hand;");
Button updateBtn = new Button("Update");
updateBtn.setStyle("-fx-background-color: #2980b9; -fx-text-fill: white; -fx-cursor: hand;");
Button deleteBtn = new Button("Delete");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
Button clearBtn = new Button("Clear");
clearBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
btnBox.getChildren().addAll(addBtn, updateBtn, deleteBtn, clearBtn);
// Add button action
addBtn.setOnAction(e -> {
try {
String id = IDGenerator.generateMedicineId();
Medicine med = new Medicine(id, nameField.getText(), mfgField.getText(),
```

```
categoryBox.getValue(), batchField.getText(), expiryField.getText(),
Integer.parseInt(qtyField.getText()), // Wrapper class parsing
Double.parseDouble(priceField.getText()), // Wrapper class parsing
supplierField.getText());
inventoryManager.addMedicine(med);
refreshTable();
clearFields(idField, nameField, mfgField, categoryBox, batchField,
expiryField, qtyField, priceField, supplierField);
statusLabel.setText("Medicine added: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
} catch (Exception ex) {
statusLabel.setText("Error: " + ex.getMessage());
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
});
// Update button action
updateBtn.setOnAction(e -> {
try {
String id = idField.getText();
if (id.isEmpty()) {
statusLabel.setText("Select a medicine first!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
Medicine med = new Medicine(id, nameField.getText(), mfgField.getText(),
categoryBox.getValue(), batchField.getText(), expiryField.getText(),
Integer.parseInt(qtyField.getText()),
Double.parseDouble(priceField.getText()),
supplierField.getText());
inventoryManager.updateMedicine(med);
refreshTable();
statusLabel.setText("Medicine updated: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
} catch (Exception ex) {
statusLabel.setText("Error: " + ex.getMessage());
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
});
// Delete button action
deleteBtn.setOnAction(e -> {
String id = idField.getText();
if (!id.isEmpty()) {
inventoryManager.deleteMedicine(id);
refreshTable();
clearFields(idField, nameField, mfgField, categoryBox, batchField,
expiryField, qtyField, priceField, supplierField);
statusLabel.setText("Medicine deleted: " + id);
}
});
// Clear button
clearBtn.setOnAction(e -> {
clearFields(idField, nameField, mfgField, categoryBox, batchField,
expiryField, qtyField, priceField, supplierField);
statusLabel.setText("");
});
// Table selection listener - populate form
```

```
tableView = new TableView<>();
tableView.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) -> {
if (newVal != null) {
idField.setText(newVal.getMedicineId());
nameField.setText(newVal.getName());
mfgField.setText(newVal.getManufacturer());
categoryBox.setValue(newVal.getCategory());
batchField.setText(newVal.getBatchNumber());
expiryField.setText(newVal.getExpiryDate());
qtyField.setText(String.valueOf(newVal.getQuantity()));
priceField.setText(String.valueOf(newVal.getPrice()));
supplierField.setText(newVal.getSupplierId());
}
});
formBox.getChildren().addAll(formTitle, grid, btnBox, statusLabel);
return formBox;
}
@SuppressWarnings("unchecked")
private VBox createTable() {
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
// Search and sort controls
HBox controlsBox = new HBox(10);
controlsBox.setAlignment(Pos.CENTER_LEFT);
TextField searchField = new TextField();
searchField.setPromptText("Search by name...");
searchField.setPrefWidth(200);
Button searchBtn = new Button("Search");
searchBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;");
searchBtn.setOnAction(e -> {
String query = searchField.getText().trim();
if (query.isEmpty()) {
refreshTable();
} else {
ArrayList<Medicine> results = inventoryManager.searchByName(query);
tableData = FXCollections.observableArrayList(results);
tableView.setItems(tableData);
}
});
ComboBox<String> sortBox = new ComboBox<>();
sortBox.getItems().addAll("Sort by Name", "Sort by Price", "Sort by Expiry");
sortBox.setPromptText("Sort by...");
sortBox.setOnAction(e -> {
String selected = sortBox.getValue();
ArrayList<Medicine> sorted;
if ("Sort by Name".equals(selected)) {
sorted = inventoryManager.getSortedByName();
} else if ("Sort by Price".equals(selected)) {
sorted = inventoryManager.getSortedByPrice();
} else {
sorted = inventoryManager.getSortedByExpiry();
}
tableData = FXCollections.observableArrayList(sorted);
tableView.setItems(tableData);
});
ComboBox<MedicineCategory> filterBox = new ComboBox<>();
```

```
filterBox.getItems().addAll(MedicineCategory.values());
filterBox.setPromptText("Filter by Category");
filterBox.setOnAction(e -> {
MedicineCategory cat = filterBox.getValue();
if (cat != null) {
ArrayList<Medicine> filtered = inventoryManager.filterByCategory(cat);
tableData = FXCollections.observableArrayList(filtered);
tableView.setItems(tableData);
}
});
Button refreshBtn = new Button("Show All");
refreshBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
refreshBtn.setOnAction(e -> refreshTable());
controlsBox.getChildren().addAll(searchField, searchBtn, sortBox, filterBox, refreshBtn);
// Table columns
TableColumn<Medicine, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("medicineId"));
idCol.setPrefWidth(80);
TableColumn<Medicine, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(150);
TableColumn<Medicine, MedicineCategory> catCol = new TableColumn<>("Category");
catCol.setCellValueFactory(new PropertyValueFactory<>("category"));
catCol.setPrefWidth(80);
TableColumn<Medicine, String> batchCol = new TableColumn<>("Batch");
batchCol.setCellValueFactory(new PropertyValueFactory<>("batchNumber"));
batchCol.setPrefWidth(70);
TableColumn<Medicine, String> expiryCol = new TableColumn<>("Expiry");
expiryCol.setCellValueFactory(new PropertyValueFactory<>("expiryDate"));
expiryCol.setPrefWidth(90);
TableColumn<Medicine, Integer> qtyCol = new TableColumn<>("Qty");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
qtyCol.setPrefWidth(50);
TableColumn<Medicine, Double> priceCol = new TableColumn<>("Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("price"));
priceCol.setPrefWidth(70);
tableView.getColumns().addAll(idCol, nameCol, catCol, batchCol, expiryCol, qtyCol, priceCol);
refreshTable();
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(controlsBox, tableView);
return tableBox;
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(inventoryManager.getAllMedicines());
tableView.setItems(tableData);
}
private void clearFields(TextField idField, TextField nameField, TextField mfgField,
ComboBox<MedicineCategory> categoryBox, TextField batchField,
TextField expiryField, TextField qtyField, TextField priceField,
TextField supplierField) {
idField.clear();
nameField.clear();
mfgField.clear();
categoryBox.setValue(null);
batchField.clear();
```

```
expiryField.clear();
qtyField.clear();
priceField.clear();
supplierField.clear();
tableView.getSelectionModel().clearSelection();
}
public Scene getScene() {
return scene;
}
}
```

25. BillingScreen.java

The left panel of this screen is where a new bill gets built up piece by piece. Typing a customer ID causes the name to appear automatically. Each medicine is added by ID and quantity, and the running total updates after each addition. When the cashier clicks the generate button, the bill is handed off to a background worker so the interface stays usable during processing. The right panel shows the current bill items and a history of past bills.

Code for BillingScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.*;
import com.pharmacy.service.*;
import com.pharmacy.util.IDGenerator;
import javafx.application.Platform;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.text.SimpleDateFormat;
import java.util.Date;
/**
* Sales and Billing Screen with multi-threaded billing simulation.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Multithreaded billing.
*/
public class BillingScreen {
private Scene scene;
private PharmacyApp app;
private BillingService billingService;
private InventoryManager inventoryManager;
private CustomerService customerService;
private UserService userService;
private TableView<BillItem> billItemsTable;
private ObservableList<BillItem> billItems;
private Label totalLabel;
private Label netLabel;
private Bill currentBill;
public BillingScreen(PharmacyApp app, BillingService billingService,
InventoryManager inventoryManager, CustomerService customerService,
```

```
UserService userService) {
this.app = app;
this.billingService = billingService;
this.inventoryManager = inventoryManager;
this.customerService = customerService;
this.userService = userService;
this.billItems = FXCollections.observableArrayList();
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(userService.getCurrentUser()));
Label titleLabel = new Label("Sales & Billing");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Bill form
VBox formBox = new VBox(12);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("New Bill");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane custGrid = new GridPane();
custGrid.setVgap(8);
custGrid.setHgap(8);
TextField custIdField = new TextField();
custIdField.setPromptText("Customer ID");
TextField custNameField = new TextField();
custNameField.setPromptText("Customer Name");
// Auto-fill customer name when ID is entered
custIdField.textProperty().addListener((obs, oldVal, newVal) -> {
if (newVal != null && !newVal.isEmpty()) {
Customer cust = customerService.getCustomerById(newVal.trim());
if (cust != null) {
custNameField.setText(cust.getName());
}
}
});
custGrid.add(new Label("Customer ID:"), 0, 0);
custGrid.add(custIdField, 1, 0);
custGrid.add(new Label("Customer Name:"), 0, 1);
custGrid.add(custNameField, 1, 1);
// Add medicine to bill
```

```
Label addItemTitle = new Label("Add Medicine to Bill");
addItemTitle.setFont(Font.font("Arial", FontWeight.BOLD, 13));
GridPane itemGrid = new GridPane();
itemGrid.setVgap(8);
itemGrid.setHgap(8);
TextField medIdField = new TextField();
medIdField.setPromptText("Medicine ID");
Label medNameLabel = new Label("-");
Label medPriceLabel = new Label("-");
Label medStockLabel = new Label("-");
TextField medQtyField = new TextField();
medQtyField.setPromptText("Quantity");
// Auto-fill medicine details
medIdField.textProperty().addListener((obs, oldVal, newVal) -> {
if (newVal != null && !newVal.isEmpty()) {
Medicine med = inventoryManager.getMedicineById(newVal.trim());
if (med != null) {
medNameLabel.setText(med.getName());
medPriceLabel.setText("Rs." + String.format("%.2f", med.getPrice()));
medStockLabel.setText("Stock: " + med.getQuantity());
}
}
});
int r = 0;
itemGrid.add(new Label("Medicine ID:"), 0, r); itemGrid.add(medIdField, 1, r++);
itemGrid.add(new Label("Name:"), 0, r); itemGrid.add(medNameLabel, 1, r++);
itemGrid.add(new Label("Price:"), 0, r); itemGrid.add(medPriceLabel, 1, r++);
itemGrid.add(new Label("Available:"), 0, r); itemGrid.add(medStockLabel, 1, r++);
itemGrid.add(new Label("Quantity:"), 0, r); itemGrid.add(medQtyField, 1, r++);
Label statusLabel = new Label();
Button addItemBtn = new Button("Add to Bill");
addItemBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;
-fx-pref-width: 200;");
addItemBtn.setOnAction(e -> {
try {
String medId = medIdField.getText().trim();
Medicine med = inventoryManager.getMedicineById(medId);
if (med == null) {
statusLabel.setText("Medicine not found!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
int qty = Integer.parseInt(medQtyField.getText().trim());
if (qty > med.getQuantity()) {
statusLabel.setText("Insufficient stock!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
BillItem item = new BillItem(medId, med.getName(), qty, med.getPrice());
billItems.add(item);
updateTotals();
medIdField.clear();
medQtyField.clear();
medNameLabel.setText("-");
medPriceLabel.setText("-");
medStockLabel.setText("-");
```

```
statusLabel.setText("Item added!");
statusLabel.setStyle("-fx-text-fill: #27ae60;");
} catch (NumberFormatException ex) {
statusLabel.setText("Invalid quantity!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
});
// Totals
totalLabel = new Label("Total: Rs.0.00");
totalLabel.setFont(Font.font("Arial", FontWeight.BOLD, 14));
TextField discountField = new TextField("0");
discountField.setPromptText("Discount %");
discountField.setPrefWidth(80);
netLabel = new Label("Net: Rs.0.00");
netLabel.setFont(Font.font("Arial", FontWeight.BOLD, 16));
netLabel.setStyle("-fx-text-fill: #27ae60;");
HBox totalsBox = new HBox(10);
totalsBox.setAlignment(Pos.CENTER_LEFT);
totalsBox.getChildren().addAll(new Label("Discount %:"), discountField);
// Generate Bill button - uses multithreaded billing
Button generateBtn = new Button("Generate Bill (Threaded)");
generateBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; " +
"-fx-font-size: 14; -fx-pref-width: 280; -fx-cursor: hand;");
generateBtn.setOnAction(e -> {
if (billItems.isEmpty()) {
statusLabel.setText("Add items first!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
if (custNameField.getText().trim().isEmpty()) {
statusLabel.setText("Enter customer name!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
String billId = IDGenerator.generateBillId();
String date = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
currentBill = new Bill(billId, custIdField.getText().trim(),
custNameField.getText().trim(), date,
userService.getCurrentUser().getUsername());
for (BillItem item : billItems) {
currentBill.addItem(item);
}
try {
double discPct = Double.parseDouble(discountField.getText().trim());
currentBill.applyDiscount(discPct); // Method overloading demo
} catch (NumberFormatException ex) {
// No discount
}
// Multi-threaded billing using BillingThread
statusLabel.setText("Processing bill...");
statusLabel.setStyle("-fx-text-fill: #3498db;");
generateBtn.setDisable(true);
BillingThread billingThread = new BillingThread(billingService, currentBill,
(success, message, bill) -> {
// Use Platform.runLater to update UI from thread
```

```
Platform.runLater(() -> {
if (success) {
statusLabel.setText(message + " Invoice generated!");
statusLabel.setStyle("-fx-text-fill: #27ae60;");
billItems.clear();
updateTotals();
custIdField.clear();
custNameField.clear();
} else {
statusLabel.setText(message);
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
generateBtn.setDisable(false);
});
});
billingThread.start();
// Demonstrate join() - wait for billing thread in another thread
Thread waitThread = new Thread(() -> {
try {
billingThread.join(); // Wait for billing to complete
System.out.println("Billing thread completed via join()");
} catch (InterruptedException ex) {
Thread.currentThread().interrupt();
}
});
waitThread.setDaemon(true);
waitThread.start();
});
Button clearBillBtn = new Button("Clear Bill");
clearBillBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor:
hand;");
clearBillBtn.setOnAction(e -> {
billItems.clear();
updateTotals();
statusLabel.setText("");
});
formBox.getChildren().addAll(formTitle, custGrid,
new Separator(), addItemTitle, itemGrid, addItemBtn, statusLabel,
new Separator(), totalLabel, totalsBox, netLabel,
generateBtn, clearBillBtn);
root.setLeft(formBox);
// Center - Bill items table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
Label tableTitle = new Label("Current Bill Items");
tableTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
billItemsTable = new TableView<>();
billItemsTable.setItems(billItems);
TableColumn<BillItem, String> nameCol = new TableColumn<>("Medicine");
nameCol.setCellValueFactory(new PropertyValueFactory<>("medicineName"));
nameCol.setPrefWidth(200);
TableColumn<BillItem, Integer> qtyCol = new TableColumn<>("Qty");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
qtyCol.setPrefWidth(60);
TableColumn<BillItem, Double> priceCol = new TableColumn<>("Unit Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("unitPrice"));
```

```
priceCol.setPrefWidth(100);
TableColumn<BillItem, Double> subCol = new TableColumn<>("Subtotal");
subCol.setCellValueFactory(new PropertyValueFactory<>("subtotal"));
subCol.setPrefWidth(100);
billItemsTable.getColumns().addAll(nameCol, qtyCol, priceCol, subCol);
// Past bills table
Label pastTitle = new Label("Past Bills");
pastTitle.setFont(Font.font("Arial", FontWeight.BOLD, 13));
TableView<Bill> pastBillsTable = new TableView<>();
ObservableList<Bill> pastBills =
FXCollections.observableArrayList(billingService.getAllBills());
pastBillsTable.setItems(pastBills);
TableColumn<Bill, String> billIdCol = new TableColumn<>("Bill ID");
billIdCol.setCellValueFactory(new PropertyValueFactory<>("billId"));
TableColumn<Bill, String> billCustCol = new TableColumn<>("Customer");
billCustCol.setCellValueFactory(new PropertyValueFactory<>("customerName"));
TableColumn<Bill, String> billDateCol = new TableColumn<>("Date");
billDateCol.setCellValueFactory(new PropertyValueFactory<>("date"));
TableColumn<Bill, Double> billAmtCol = new TableColumn<>("Amount");
billAmtCol.setCellValueFactory(new PropertyValueFactory<>("netAmount"));
pastBillsTable.getColumns().addAll(billIdCol, billCustCol, billDateCol, billAmtCol);
pastBillsTable.setPrefHeight(200);
VBox.setVgrow(billItemsTable, Priority.ALWAYS);
tableBox.getChildren().addAll(tableTitle, billItemsTable, pastTitle, pastBillsTable);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void updateTotals() {
double total = 0;
for (BillItem item : billItems) {
total += item.getSubtotal();
}
totalLabel.setText(String.format("Total: Rs.%.2f", total));
netLabel.setText(String.format("Net: Rs.%.2f", total));
}
public Scene getScene() {
return scene;
}
}
```

26. SupplierScreen.java

A table of supplier records on the right, a form on the left, and four action buttons in between. Selecting a table row fills the form. Clicking Add creates a new record with a freshly generated ID. Update and Delete act on whatever is currently selected. A search field above the table filters by company name.

Code for SupplierScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Supplier;
import com.pharmacy.service.SupplierService;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
```

```
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Supplier Management Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Event handling.
*/
public class SupplierScreen {
private Scene scene;
private PharmacyApp app;
private SupplierService supplierService;
private TableView<Supplier> tableView;
private ObservableList<Supplier> tableData;
public SupplierScreen(PharmacyApp app, SupplierService supplierService) {
this.app = app;
this.supplierService = supplierService;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Supplier Management");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Form
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("Add / Edit Supplier");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane grid = new GridPane();
grid.setVgap(8);
grid.setHgap(8);
TextField idField = new TextField();
idField.setEditable(false);
idField.setPromptText("Auto-generated");
TextField companyField = new TextField();
```

```
companyField.setPromptText("Company name");
TextField contactField = new TextField();
contactField.setPromptText("Contact person");
TextField phoneField = new TextField();
phoneField.setPromptText("Phone");
TextField emailField = new TextField();
emailField.setPromptText("Email");
TextField addressField = new TextField();
addressField.setPromptText("Address");
int row = 0;
grid.add(new Label("ID:"), 0, row); grid.add(idField, 1, row++);
grid.add(new Label("Company:"), 0, row); grid.add(companyField, 1, row++);
grid.add(new Label("Contact:"), 0, row); grid.add(contactField, 1, row++);
grid.add(new Label("Phone:"), 0, row); grid.add(phoneField, 1, row++);
grid.add(new Label("Email:"), 0, row); grid.add(emailField, 1, row++);
grid.add(new Label("Address:"), 0, row); grid.add(addressField, 1, row++);
Label statusLabel = new Label();
HBox btnBox = new HBox(8);
Button addBtn = new Button("Add");
addBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-cursor: hand;");
Button updateBtn = new Button("Update");
updateBtn.setStyle("-fx-background-color: #2980b9; -fx-text-fill: white; -fx-cursor: hand;");
Button deleteBtn = new Button("Delete");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
Button clearBtn = new Button("Clear");
clearBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
btnBox.getChildren().addAll(addBtn, updateBtn, deleteBtn, clearBtn);
addBtn.setOnAction(e -> {
String id = IDGenerator.generateSupplierId();
Supplier sup = new Supplier(id, companyField.getText(), contactField.getText(),
phoneField.getText(), emailField.getText(), addressField.getText());
supplierService.addSupplier(sup);
refreshTable();
clearFields(idField, companyField, contactField, phoneField, emailField, addressField);
statusLabel.setText("Supplier added: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
updateBtn.setOnAction(e -> {
String id = idField.getText();
if (id.isEmpty()) { statusLabel.setText("Select a supplier!"); return; }
Supplier sup = new Supplier(id, companyField.getText(), contactField.getText(),
phoneField.getText(), emailField.getText(), addressField.getText());
supplierService.updateSupplier(sup);
refreshTable();
statusLabel.setText("Supplier updated: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
deleteBtn.setOnAction(e -> {
String id = idField.getText();
if (!id.isEmpty()) {
supplierService.deleteSupplier(id);
refreshTable();
clearFields(idField, companyField, contactField, phoneField, emailField, addressField);
statusLabel.setText("Supplier deleted.");
}
});
```

```
clearBtn.setOnAction(e -> {
clearFields(idField, companyField, contactField, phoneField, emailField, addressField);
statusLabel.setText("");
});
formBox.getChildren().addAll(formTitle, grid, btnBox, statusLabel);
root.setLeft(formBox);
// Center - Table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
// Search
HBox searchBox = new HBox(10);
TextField searchField = new TextField();
searchField.setPromptText("Search by company name...");
Button searchBtn = new Button("Search");
searchBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;");
searchBtn.setOnAction(e -> {
String q = searchField.getText().trim();
if (q.isEmpty()) refreshTable();
else {
tableData = FXCollections.observableArrayList(supplierService.searchByName(q));
tableView.setItems(tableData);
}
});
Button showAllBtn = new Button("Show All");
showAllBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
showAllBtn.setOnAction(e -> refreshTable());
searchBox.getChildren().addAll(searchField, searchBtn, showAllBtn);
tableView = new TableView<>();
TableColumn<Supplier, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("supplierId"));
TableColumn<Supplier, String> compCol = new TableColumn<>("Company");
compCol.setCellValueFactory(new PropertyValueFactory<>("companyName"));
compCol.setPrefWidth(150);
TableColumn<Supplier, String> contCol = new TableColumn<>("Contact");
contCol.setCellValueFactory(new PropertyValueFactory<>("contactPerson"));
contCol.setPrefWidth(120);
TableColumn<Supplier, String> phoneCol = new TableColumn<>("Phone");
phoneCol.setCellValueFactory(new PropertyValueFactory<>("phone"));
phoneCol.setPrefWidth(110);
TableColumn<Supplier, String> emailCol = new TableColumn<>("Email");
emailCol.setCellValueFactory(new PropertyValueFactory<>("email"));
emailCol.setPrefWidth(150);
tableView.getColumns().addAll(idCol, compCol, contCol, phoneCol, emailCol);
refreshTable();
// Table selection listener
tableView.getSelectionModel().selectedItemProperty().addListener((obs, o, n) -> {
if (n != null) {
idField.setText(n.getSupplierId());
companyField.setText(n.getCompanyName());
contactField.setText(n.getContactPerson());
phoneField.setText(n.getPhone());
emailField.setText(n.getEmail());
addressField.setText(n.getAddress());
}
});
```

```
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(searchBox, tableView);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(supplierService.getAllSuppliers());
tableView.setItems(tableData);
}
private void clearFields(TextField... fields) {
for (TextField f : fields) f.clear();
tableView.getSelectionModel().clearSelection();
}
public Scene getScene() { return scene; }
}
```

27. CustomerScreen.java

Laid out the same way as the supplier screen but dealing with patient records. The search runs against the patient name field. IDs are generated automatically in the CUS-prefixed format. The form clears itself and deselects the table row when the Clear button is pressed.

Code for CustomerScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Customer;
import com.pharmacy.service.CustomerService;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Customer/Patient Records Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Event handling.
*/
public class CustomerScreen {
private Scene scene;
private PharmacyApp app;
private CustomerService customerService;
private TableView<Customer> tableView;
private ObservableList<Customer> tableData;
public CustomerScreen(PharmacyApp app, CustomerService customerService) {
this.app = app;
this.customerService = customerService;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
```

```
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Customer / Patient Records");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Form
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("Add / Edit Customer");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane grid = new GridPane();
grid.setVgap(8);
grid.setHgap(8);
TextField idField = new TextField();
idField.setEditable(false);
idField.setPromptText("Auto-generated");
TextField nameField = new TextField();
nameField.setPromptText("Full name");
TextField phoneField = new TextField();
phoneField.setPromptText("Phone number");
TextField emailField = new TextField();
emailField.setPromptText("Email");
TextField addressField = new TextField();
addressField.setPromptText("Address");
int row = 0;
grid.add(new Label("ID:"), 0, row); grid.add(idField, 1, row++);
grid.add(new Label("Name:"), 0, row); grid.add(nameField, 1, row++);
grid.add(new Label("Phone:"), 0, row); grid.add(phoneField, 1, row++);
grid.add(new Label("Email:"), 0, row); grid.add(emailField, 1, row++);
grid.add(new Label("Address:"), 0, row); grid.add(addressField, 1, row++);
Label statusLabel = new Label();
HBox btnBox = new HBox(8);
Button addBtn = new Button("Add");
addBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-cursor: hand;");
Button updateBtn = new Button("Update");
updateBtn.setStyle("-fx-background-color: #2980b9; -fx-text-fill: white; -fx-cursor: hand;");
Button deleteBtn = new Button("Delete");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
Button clearBtn = new Button("Clear");
clearBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
btnBox.getChildren().addAll(addBtn, updateBtn, deleteBtn, clearBtn);
addBtn.setOnAction(e -> {
String id = IDGenerator.generateCustomerId();
```

```
Customer cust = new Customer(id, nameField.getText(), phoneField.getText(),
emailField.getText(), addressField.getText());
customerService.addCustomer(cust);
refreshTable();
clearFields(idField, nameField, phoneField, emailField, addressField);
statusLabel.setText("Customer added: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
updateBtn.setOnAction(e -> {
String id = idField.getText();
if (id.isEmpty()) { statusLabel.setText("Select a customer!"); return; }
Customer cust = new Customer(id, nameField.getText(), phoneField.getText(),
emailField.getText(), addressField.getText());
customerService.updateCustomer(cust);
refreshTable();
statusLabel.setText("Customer updated: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
deleteBtn.setOnAction(e -> {
String id = idField.getText();
if (!id.isEmpty()) {
customerService.deleteCustomer(id);
refreshTable();
clearFields(idField, nameField, phoneField, emailField, addressField);
statusLabel.setText("Customer deleted.");
}
});
clearBtn.setOnAction(e -> {
clearFields(idField, nameField, phoneField, emailField, addressField);
statusLabel.setText("");
});
formBox.getChildren().addAll(formTitle, grid, btnBox, statusLabel);
root.setLeft(formBox);
// Center - Table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
HBox searchBox = new HBox(10);
TextField searchField = new TextField();
searchField.setPromptText("Search by name...");
Button searchBtn = new Button("Search");
searchBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;");
searchBtn.setOnAction(e -> {
String q = searchField.getText().trim();
if (q.isEmpty()) refreshTable();
else {
tableData = FXCollections.observableArrayList(customerService.searchByName(q));
tableView.setItems(tableData);
}
});
Button showAllBtn = new Button("Show All");
showAllBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
showAllBtn.setOnAction(e -> refreshTable());
searchBox.getChildren().addAll(searchField, searchBtn, showAllBtn);
tableView = new TableView<>();
TableColumn<Customer, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("customerId"));
```

```
TableColumn<Customer, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(150);
TableColumn<Customer, String> phoneCol = new TableColumn<>("Phone");
phoneCol.setCellValueFactory(new PropertyValueFactory<>("phone"));
phoneCol.setPrefWidth(120);
TableColumn<Customer, String> emailCol = new TableColumn<>("Email");
emailCol.setCellValueFactory(new PropertyValueFactory<>("email"));
emailCol.setPrefWidth(170);
TableColumn<Customer, String> addrCol = new TableColumn<>("Address");
addrCol.setCellValueFactory(new PropertyValueFactory<>("address"));
addrCol.setPrefWidth(180);
tableView.getColumns().addAll(idCol, nameCol, phoneCol, emailCol, addrCol);
refreshTable();
tableView.getSelectionModel().selectedItemProperty().addListener((obs, o, n) -> {
if (n != null) {
idField.setText(n.getCustomerId());
nameField.setText(n.getName());
phoneField.setText(n.getPhone());
emailField.setText(n.getEmail());
addressField.setText(n.getAddress());
}
});
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(searchBox, tableView);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(customerService.getAllCustomers());
tableView.setItems(tableData);
}
private void clearFields(TextField... fields) {
for (TextField f : fields) f.clear();
tableView.getSelectionModel().clearSelection();
}
public Scene getScene() { return scene; }
}
```

28. PrescriptionScreen.java

This screen is a step more involved than the others because one prescription contains multiple medicine entries. After filling in the patient and doctor details, the staff member adds medicines one at a time - each addition appears in a list on screen. Saving commits the whole prescription as a single record. Clicking an existing prescription in the table populates a text area below with the complete details including every prescribed medicine.

Code for PrescriptionScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.*;
import com.pharmacy.service.*;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
```

```
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.text.SimpleDateFormat;
import java.util.Date;
/**
* Prescription Handling Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, ArrayList operations.
*/
public class PrescriptionScreen {
private Scene scene;
private PharmacyApp app;
private PrescriptionService prescriptionService;
private CustomerService customerService;
private InventoryManager inventoryManager;
private TableView<Prescription> tableView;
private ObservableList<Prescription> tableData;
public PrescriptionScreen(PharmacyApp app, PrescriptionService prescriptionService,
CustomerService customerService, InventoryManager inventoryManager) {
this.app = app;
this.prescriptionService = prescriptionService;
this.customerService = customerService;
this.inventoryManager = inventoryManager;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Prescription Handling");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Form
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(330);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("New Prescription");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane grid = new GridPane();
```

```
grid.setVgap(8);
grid.setHgap(8);
TextField custIdField = new TextField();
custIdField.setPromptText("Customer ID");
TextField custNameField = new TextField();
custNameField.setPromptText("Patient name");
TextField doctorField = new TextField();
doctorField.setPromptText("Doctor name");
TextField notesField = new TextField();
notesField.setPromptText("Notes/Instructions");
custIdField.textProperty().addListener((obs, o, n) -> {
if (n != null && !n.isEmpty()) {
Customer cust = customerService.getCustomerById(n.trim());
if (cust != null) custNameField.setText(cust.getName());
}
});
int r = 0;
grid.add(new Label("Customer ID:"), 0, r); grid.add(custIdField, 1, r++);
grid.add(new Label("Patient Name:"), 0, r); grid.add(custNameField, 1, r++);
grid.add(new Label("Doctor:"), 0, r); grid.add(doctorField, 1, r++);
grid.add(new Label("Notes:"), 0, r); grid.add(notesField, 1, r++);
// Prescription items
Label itemsTitle = new Label("Prescribed Medicines");
itemsTitle.setFont(Font.font("Arial", FontWeight.BOLD, 13));
TextField medIdField = new TextField();
medIdField.setPromptText("Medicine ID");
Label medNameLabel = new Label("-");
TextField medQtyField = new TextField();
medQtyField.setPromptText("Quantity");
TextField dosageField = new TextField();
dosageField.setPromptText("e.g., 1-0-1");
medIdField.textProperty().addListener((obs, o, n) -> {
if (n != null && !n.isEmpty()) {
Medicine med = inventoryManager.getMedicineById(n.trim());
if (med != null) medNameLabel.setText(med.getName());
}
});
GridPane itemGrid = new GridPane();
itemGrid.setVgap(6);
itemGrid.setHgap(6);
int ir = 0;
itemGrid.add(new Label("Med ID:"), 0, ir); itemGrid.add(medIdField, 1, ir++);
itemGrid.add(new Label("Name:"), 0, ir); itemGrid.add(medNameLabel, 1, ir++);
itemGrid.add(new Label("Qty:"), 0, ir); itemGrid.add(medQtyField, 1, ir++);
itemGrid.add(new Label("Dosage:"), 0, ir); itemGrid.add(dosageField, 1, ir++);
// Items list display
ListView<String> itemsList = new ListView<>();
itemsList.setPrefHeight(120);
java.util.ArrayList<PrescriptionItem> presItems = new java.util.ArrayList<>();
Label statusLabel = new Label();
Button addItemBtn = new Button("Add Medicine");
addItemBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;
-fx-pref-width: 200;");
addItemBtn.setOnAction(e -> {
try {
```

```
String medId = medIdField.getText().trim();
Medicine med = inventoryManager.getMedicineById(medId);
if (med == null) { statusLabel.setText("Medicine not found!"); return; }
int qty = Integer.parseInt(medQtyField.getText().trim());
PrescriptionItem item = new PrescriptionItem(medId, med.getName(), qty, dosageField.getText());
presItems.add(item);
itemsList.getItems().add(item.toString());
medIdField.clear();
medQtyField.clear();
dosageField.clear();
medNameLabel.setText("-");
} catch (Exception ex) {
statusLabel.setText("Error: " + ex.getMessage());
}
});
Button saveBtn = new Button("Save Prescription");
saveBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-font-size: 13; " +
"-fx-pref-width: 280; -fx-cursor: hand;");
saveBtn.setOnAction(e -> {
if (custNameField.getText().trim().isEmpty() || doctorField.getText().trim().isEmpty()) {
statusLabel.setText("Fill all fields!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
String presId = IDGenerator.generatePrescriptionId();
String date = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
Prescription pres = new Prescription(presId, custIdField.getText().trim(),
custNameField.getText(), doctorField.getText(), date, notesField.getText());
for (PrescriptionItem item : presItems) {
pres.addItem(item);
}
prescriptionService.addPrescription(pres);
refreshTable();
presItems.clear();
itemsList.getItems().clear();
custIdField.clear(); custNameField.clear(); doctorField.clear(); notesField.clear();
statusLabel.setText("Prescription saved: " + presId);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
formBox.getChildren().addAll(formTitle, grid, new Separator(),
itemsTitle, itemGrid, addItemBtn, itemsList, new Separator(),
saveBtn, statusLabel);
root.setLeft(formBox);
// Center - Table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
Label tableTitle = new Label("All Prescriptions");
tableTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
tableView = new TableView<>();
TableColumn<Prescription, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("prescriptionId"));
TableColumn<Prescription, String> patCol = new TableColumn<>("Patient");
patCol.setCellValueFactory(new PropertyValueFactory<>("customerName"));
patCol.setPrefWidth(130);
TableColumn<Prescription, String> docCol = new TableColumn<>("Doctor");
docCol.setCellValueFactory(new PropertyValueFactory<>("doctorName"));
```

```
docCol.setPrefWidth(130);
TableColumn<Prescription, String> dateCol = new TableColumn<>("Date");
dateCol.setCellValueFactory(new PropertyValueFactory<>("date"));
TableColumn<Prescription, String> notesCol = new TableColumn<>("Notes");
notesCol.setCellValueFactory(new PropertyValueFactory<>("notes"));
notesCol.setPrefWidth(180);
tableView.getColumns().addAll(idCol, patCol, docCol, dateCol, notesCol);
refreshTable();
// Details area
TextArea detailsArea = new TextArea();
detailsArea.setEditable(false);
detailsArea.setPrefHeight(150);
detailsArea.setPromptText("Select a prescription to view details...");
tableView.getSelectionModel().selectedItemProperty().addListener((obs, o, n) -> {
if (n != null) {
StringBuilder sb = new StringBuilder();
sb.append("Prescription: ").append(n.getPrescriptionId()).append("\n");
sb.append("Patient: ").append(n.getCustomerName()).append("\n");
sb.append("Doctor: Dr. ").append(n.getDoctorName()).append("\n");
sb.append("Date: ").append(n.getDate()).append("\n");
sb.append("Notes: ").append(n.getNotes()).append("\n\n");
sb.append("Prescribed Medicines:\n");
for (PrescriptionItem item : n.getItems()) {
sb.append(" - ").append(item.toString()).append("\n");
}
detailsArea.setText(sb.toString());
}
});
Button deleteBtn = new Button("Delete Selected");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
deleteBtn.setOnAction(e -> {
Prescription sel = tableView.getSelectionModel().getSelectedItem();
if (sel != null) {
prescriptionService.deletePrescription(sel.getPrescriptionId());
refreshTable();
detailsArea.clear();
}
});
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(tableTitle, tableView, new Label("Details:"), detailsArea,
deleteBtn);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(prescriptionService.getAllPrescriptions());
tableView.setItems(tableData);
}
public Scene getScene() { return scene; }
}
```

29. ReportsScreen.java

A navigation panel on the left switches between different views. The landing view shows summary cards - coloured boxes each displaying one key number like total medicines or total revenue. Other

views show the full transaction history, the complete stock list, medicines nearing expiry, and medicines running low. The backup button starts the backup worker in a fresh thread and shows a spinning indicator until the callback fires with a completion message.

Code for ReportsScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Bill;
import com.pharmacy.model.Medicine;
import com.pharmacy.service.*;
import javafx.application.Platform;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
/**
* Reports & Alerts Screen - Sales reports, stock alerts, expiry alerts, backups.
* Demonstrates: JavaFX TableView, VBox/HBox, Multithreading integration, Backup service.
*/
public class ReportsScreen {
private Scene scene;
private PharmacyApp app;
private InventoryManager inventoryManager;
private BillingService billingService;
private CustomerService customerService;
private SupplierService supplierService;
private ExpiryAlertThread expiryAlertThread;
private StockAlertThread stockAlertRunnable;
public ReportsScreen(PharmacyApp app, InventoryManager inventoryManager,
BillingService billingService, CustomerService customerService,
SupplierService supplierService,
ExpiryAlertThread expiryAlertThread, StockAlertThread stockAlertRunnable) {
this.app = app;
this.inventoryManager = inventoryManager;
this.billingService = billingService;
this.customerService = customerService;
this.supplierService = supplierService;
this.expiryAlertThread = expiryAlertThread;
this.stockAlertRunnable = stockAlertRunnable;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
```

```
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Reports & Alerts");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Report categories
VBox leftMenu = new VBox(8);
leftMenu.setPadding(new Insets(15));
leftMenu.setPrefWidth(200);
leftMenu.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0
1 0 0;");
Label menuTitle = new Label("Report Types");
menuTitle.setFont(Font.font("Arial", FontWeight.BOLD, 14));
// Content area in center
VBox contentArea = new VBox(10);
contentArea.setPadding(new Insets(15));
Button salesReportBtn = createMenuButton("Sales Report", "#27ae60");
Button stockReportBtn = createMenuButton("Stock Report", "#3498db");
Button expiryAlertBtn = createMenuButton("Expiry Alerts", "#e74c3c");
Button lowStockBtn = createMenuButton("Low Stock Alert", "#e67e22");
Button summaryBtn = createMenuButton("Summary", "#9b59b6");
Button backupBtn = createMenuButton("Backup Data", "#34495e");
// Sales Report
salesReportBtn.setOnAction(e -> showSalesReport(contentArea));
// Stock Report
stockReportBtn.setOnAction(e -> showStockReport(contentArea));
// Expiry Alerts (from alert thread)
expiryAlertBtn.setOnAction(e -> showExpiryAlerts(contentArea));
// Low Stock Alert (from stock thread)
lowStockBtn.setOnAction(e -> showLowStockAlerts(contentArea));
// Summary
summaryBtn.setOnAction(e -> showSummary(contentArea));
// Backup
backupBtn.setOnAction(e -> runBackup(contentArea));
leftMenu.getChildren().addAll(menuTitle, new Separator(),
salesReportBtn, stockReportBtn, expiryAlertBtn,
lowStockBtn, summaryBtn, new Separator(), backupBtn);
root.setLeft(leftMenu);
// Initial content
showSummary(contentArea);
root.setCenter(contentArea);
scene = new Scene(root, 1100, 700);
}
private void showSalesReport(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Sales Report");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
```

```
String today = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
Label totalSales = new Label(String.format("Total Sales (All Time): Rs.%.2f",
billingService.getTotalSales()));
totalSales.setFont(Font.font("Arial", FontWeight.BOLD, 14));
totalSales.setStyle("-fx-text-fill: #27ae60;");
Label todaySales = new Label(String.format("Today's Sales: Rs.%.2f",
billingService.getTotalSalesByDate(today)));
todaySales.setFont(Font.font("Arial", FontWeight.BOLD, 14));
Label billCount = new Label("Total Bills: " + billingService.getBillCount());
// Bills table
TableView<Bill> billTable = new TableView<>();
ObservableList<Bill> bills = FXCollections.observableArrayList(billingService.getAllBills());
billTable.setItems(bills);
TableColumn<Bill, String> idCol = new TableColumn<>("Bill ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("billId"));
TableColumn<Bill, String> custCol = new TableColumn<>("Customer");
custCol.setCellValueFactory(new PropertyValueFactory<>("customerName"));
custCol.setPrefWidth(150);
TableColumn<Bill, String> dateCol = new TableColumn<>("Date");
dateCol.setCellValueFactory(new PropertyValueFactory<>("date"));
TableColumn<Bill, Double> totalCol = new TableColumn<>("Total");
totalCol.setCellValueFactory(new PropertyValueFactory<>("totalAmount"));
TableColumn<Bill, Double> discCol = new TableColumn<>("Discount");
discCol.setCellValueFactory(new PropertyValueFactory<>("discount"));
TableColumn<Bill, Double> netCol = new TableColumn<>("Net Amount");
netCol.setCellValueFactory(new PropertyValueFactory<>("netAmount"));
netCol.setPrefWidth(100);
TableColumn<Bill, String> byCol = new TableColumn<>("Billed By");
byCol.setCellValueFactory(new PropertyValueFactory<>("billedBy"));
billTable.getColumns().addAll(idCol, custCol, dateCol, totalCol, discCol, netCol, byCol);
VBox.setVgrow(billTable, Priority.ALWAYS);
contentArea.getChildren().addAll(title, totalSales, todaySales, billCount, billTable);
}
@SuppressWarnings("unchecked")
private void showStockReport(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Stock Report");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
Label countLabel = new Label("Total Medicines in Inventory: " +
inventoryManager.getMedicineCount());
countLabel.setFont(Font.font("Arial", FontWeight.BOLD, 14));
TableView<Medicine> table = new TableView<>();
ObservableList<Medicine> data =
FXCollections.observableArrayList(inventoryManager.getSortedByName());
table.setItems(data);
TableColumn<Medicine, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("medicineId"));
TableColumn<Medicine, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(170);
TableColumn<Medicine, String> catCol = new TableColumn<>("Category");
catCol.setCellValueFactory(new PropertyValueFactory<>("category"));
TableColumn<Medicine, Integer> qtyCol = new TableColumn<>("Quantity");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
TableColumn<Medicine, Double> priceCol = new TableColumn<>("Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("price"));
```

```
TableColumn<Medicine, String> expCol = new TableColumn<>("Expiry");
expCol.setCellValueFactory(new PropertyValueFactory<>("expiryDate"));
expCol.setPrefWidth(100);
table.getColumns().addAll(idCol, nameCol, catCol, qtyCol, priceCol, expCol);
VBox.setVgrow(table, Priority.ALWAYS);
contentArea.getChildren().addAll(title, countLabel, table);
}
@SuppressWarnings("unchecked")
private void showExpiryAlerts(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Expiry Alerts");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
title.setStyle("-fx-text-fill: #e74c3c;");
SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
String currentDate = sdf.format(new Date());
Calendar cal = Calendar.getInstance();
cal.add(Calendar.DAY_OF_MONTH, 30);
String thresholdDate = sdf.format(cal.getTime());
// Expired medicines
ArrayList<Medicine> expired = inventoryManager.getExpiredMedicines(currentDate);
Label expiredTitle = new Label("EXPIRED Medicines (" + expired.size() + ")");
expiredTitle.setFont(Font.font("Arial", FontWeight.BOLD, 14));
expiredTitle.setStyle("-fx-text-fill: #e74c3c;");
TableView<Medicine> expiredTable = new TableView<>();
expiredTable.setItems(FXCollections.observableArrayList(expired));
addMedicineColumns(expiredTable);
expiredTable.setPrefHeight(200);
// Expiring soon
ArrayList<Medicine> expiringSoon = inventoryManager.getExpiringSoon(currentDate,
thresholdDate);
Label expiringTitle = new Label("Expiring Within 30 Days (" + expiringSoon.size() + ")");
expiringTitle.setFont(Font.font("Arial", FontWeight.BOLD, 14));
expiringTitle.setStyle("-fx-text-fill: #e67e22;");
TableView<Medicine> expiringTable = new TableView<>();
expiringTable.setItems(FXCollections.observableArrayList(expiringSoon));
addMedicineColumns(expiringTable);
VBox.setVgrow(expiringTable, Priority.ALWAYS);
contentArea.getChildren().addAll(title, expiredTitle, expiredTable, expiringTitle,
expiringTable);
}
@SuppressWarnings("unchecked")
private void showLowStockAlerts(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Low Stock Alerts (< 10 units)");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
title.setStyle("-fx-text-fill: #e67e22;");
ArrayList<Medicine> lowStock = inventoryManager.getLowStockMedicines();
Label countLabel = new Label("Low Stock Items: " + lowStock.size());
countLabel.setFont(Font.font("Arial", FontWeight.BOLD, 14));
TableView<Medicine> table = new TableView<>();
table.setItems(FXCollections.observableArrayList(lowStock));
addMedicineColumns(table);
VBox.setVgrow(table, Priority.ALWAYS);
contentArea.getChildren().addAll(title, countLabel, table);
}
```

```
private void showSummary(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("System Summary");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
GridPane summaryGrid = new GridPane();
summaryGrid.setVgap(15);
summaryGrid.setHgap(20);
summaryGrid.setPadding(new Insets(20));
addSummaryCard(summaryGrid, 0, 0, "Total Medicines",
String.valueOf(inventoryManager.getMedicineCount()), "#3498db");
addSummaryCard(summaryGrid, 1, 0, "Total Customers",
String.valueOf(customerService.getCustomerCount()), "#9b59b6");
addSummaryCard(summaryGrid, 2, 0, "Total Suppliers",
String.valueOf(supplierService.getSupplierCount()), "#e67e22");
addSummaryCard(summaryGrid, 0, 1, "Total Bills",
String.valueOf(billingService.getBillCount()), "#27ae60");
addSummaryCard(summaryGrid, 1, 1, "Total Sales",
String.format("Rs.%.2f", billingService.getTotalSales()), "#2c3e50");
String currentDate = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
addSummaryCard(summaryGrid, 2, 1, "Low Stock Items",
String.valueOf(inventoryManager.getLowStockMedicines().size()), "#e74c3c");
// Thread status
Label threadStatus = new Label("Background Threads:");
threadStatus.setFont(Font.font("Arial", FontWeight.BOLD, 14));
Label expiryThreadLabel = new Label("Expiry Alert Thread: " +
(expiryAlertThread != null && expiryAlertThread.isAlive() ? "RUNNING" : "STOPPED"));
expiryThreadLabel.setStyle(expiryAlertThread != null && expiryAlertThread.isAlive() ?
"-fx-text-fill: #27ae60;" : "-fx-text-fill: #e74c3c;");
Label stockThreadLabel = new Label("Stock Alert Thread: RUNNING (daemon)");
stockThreadLabel.setStyle("-fx-text-fill: #27ae60;");
contentArea.getChildren().addAll(title, summaryGrid, new Separator(),
threadStatus, expiryThreadLabel, stockThreadLabel);
}
private void addSummaryCard(GridPane grid, int col, int row, String label, String value, String
color) {
VBox card = new VBox(5);
card.setPadding(new Insets(15));
card.setPrefSize(180, 80);
card.setAlignment(Pos.CENTER);
card.setStyle("-fx-background-color: " + color + "; -fx-background-radius: 8;");
Label valLabel = new Label(value);
valLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
valLabel.setStyle("-fx-text-fill: white;");
Label nameLabel = new Label(label);
nameLabel.setStyle("-fx-text-fill: #ecf0f1; -fx-font-size: 12;");
card.getChildren().addAll(valLabel, nameLabel);
grid.add(card, col, row);
}
private void runBackup(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Data Backup");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
Label statusLabel = new Label("Starting backup...");
statusLabel.setStyle("-fx-text-fill: #3498db; -fx-font-size: 14;");
```

```
ProgressIndicator progress = new ProgressIndicator();
progress.setPrefSize(50, 50);
contentArea.getChildren().addAll(title, progress, statusLabel);
// Run backup in a separate thread (BackupService implements Runnable)
BackupService backupService = new BackupService(
inventoryManager, customerService, supplierService,
billingService, app.getPrescriptionService());
backupService.setCallback(message -> {
Platform.runLater(() -> {
statusLabel.setText(message);
statusLabel.setStyle("-fx-text-fill: #27ae60; -fx-font-size: 14;");
progress.setProgress(1.0);
Label pathLabel = new Label("Backup files saved in: data/ directory");
pathLabel.setStyle("-fx-font-size: 12;");
Label filesLabel = new Label(
"Files created:\n" +
" - backup_medicines.txt\n" +
" - backup_customers.txt\n" +
" - backup_suppliers.txt\n" +
" - backup_bills.txt\n" +
" - backup_prescriptions.txt");
filesLabel.setStyle("-fx-font-size: 12;");
contentArea.getChildren().addAll(pathLabel, filesLabel);
});
});
Thread backupThread = new Thread(backupService, "BackupThread");
backupThread.setDaemon(true);
backupThread.start();
}
@SuppressWarnings("unchecked")
private void addMedicineColumns(TableView<Medicine> table) {
TableColumn<Medicine, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("medicineId"));
TableColumn<Medicine, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(150);
TableColumn<Medicine, String> catCol = new TableColumn<>("Category");
catCol.setCellValueFactory(new PropertyValueFactory<>("category"));
TableColumn<Medicine, Integer> qtyCol = new TableColumn<>("Qty");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
TableColumn<Medicine, Double> priceCol = new TableColumn<>("Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("price"));
TableColumn<Medicine, String> expCol = new TableColumn<>("Expiry");
expCol.setCellValueFactory(new PropertyValueFactory<>("expiryDate"));
expCol.setPrefWidth(100);
table.getColumns().addAll(idCol, nameCol, catCol, qtyCol, priceCol, expCol);
}
private Button createMenuButton(String text, String color) {
Button btn = new Button(text);
btn.setPrefWidth(170);
btn.setStyle("-fx-background-color: " + color + "; -fx-text-fill: white; " +
"-fx-font-size: 12; -fx-cursor: hand; -fx-alignment: center-left;");
return btn;
}
public Scene getScene() { return scene; }
```

```
}
```

Util Package (com.pharmacy.util)

30. DataStore.java

Every service class needs to save its list to disk and load it back on startup. Rather than writing that file handling code five separate times, we wrote it once as a generic class that works with any saveable object type. The save operation wraps the list in a binary stream and writes it out as a .dat file. The load operation reverses that. A separate method writes the same data as readable text for the backup copies. All four methods carry locks to prevent a background thread from reading a file mid-write.

Code for DataStore.java:

```
package com.pharmacy.util;
import java.io.*;
import java.util.ArrayList;
/**
* Generic data store for serialization and deserialization.
* Demonstrates: Generics, Serialization/Deserialization, File I/O.
*
* @param <T> The type of objects to store (must be Serializable)
*/
public class DataStore<T extends Serializable> {
private String filePath;
public DataStore(String filePath) {
this.filePath = filePath;
}
/**
* Save a list of objects using Serialization (ObjectOutputStream).
* Demonstrates: FileOutputStream, ObjectOutputStream, Serialization.
*/
public synchronized void saveAll(ArrayList<T> items) {
try (FileOutputStream fos = new FileOutputStream(filePath);
ObjectOutputStream oos = new ObjectOutputStream(fos)) {
oos.writeObject(items);
} catch (IOException e) {
System.err.println("Error saving data to " + filePath + ": " + e.getMessage());
}
}
/**
* Load a list of objects using Deserialization (ObjectInputStream).
* Demonstrates: FileInputStream, ObjectInputStream, Deserialization.
*/
@SuppressWarnings("unchecked")
public synchronized ArrayList<T> loadAll() {
File file = new File(filePath);
if (!file.exists()) {
return new ArrayList<>();
}
try (FileInputStream fis = new FileInputStream(filePath);
ObjectInputStream ois = new ObjectInputStream(fis)) {
return (ArrayList<T>) ois.readObject();
} catch (IOException | ClassNotFoundException e) {
System.err.println("Error loading data from " + filePath + ": " + e.getMessage());
```

```
return new ArrayList<>();
}
}
/**
* Backup data to a text file using FileWriter.
* Demonstrates: FileWriter usage.
*/
public void backupToText(ArrayList<T> items, String backupPath) {
try (FileWriter writer = new FileWriter(backupPath)) {
writer.write("=== DATA BACKUP ===\n");
writer.write("Source: " + filePath + "\n");
writer.write("Records: " + items.size() + "\n");
writer.write("====================\n\n");
for (T item : items) {
writer.write(item.toString() + "\n");
}
writer.write("\n=== END OF BACKUP ===\n");
} catch (IOException e) {
System.err.println("Error creating backup: " + e.getMessage());
}
}
/**
* Read backup file content using FileReader.
* Demonstrates: FileReader usage.
*/
public String readBackup(String backupPath) {
StringBuilder content = new StringBuilder();
File file = new File(backupPath);
if (!file.exists()) {
return "No backup found.";
}
try (FileReader reader = new FileReader(backupPath)) {
int ch;
while ((ch = reader.read()) != -1) {
content.append((char) ch);
}
} catch (IOException e) {
System.err.println("Error reading backup: " + e.getMessage());
}
return content.toString();
}
public String getFilePath() {
return filePath;
}
}
```

31. IDGenerator.java

Every record type needs a unique identifier that does not repeat across sessions. This class keeps a counter for each type - medicines, customers, suppliers, bills, and prescriptions - and increments it each time a new ID is needed. The increment step is locked so two threads asking for a new ID simultaneously always get different values. On startup, the counters are nudged up to at least the highest value already present in the saved data.

Code for IDGenerator.java:

```
package com.pharmacy.util;
```

```
/**
* Utility class for generating unique IDs.
* Demonstrates: Static methods, synchronized for thread safety.
*/
public class IDGenerator {
private static int medicineCounter = 1000;
private static int customerCounter = 2000;
private static int supplierCounter = 3000;
private static int billCounter = 4000;
private static int prescriptionCounter = 5000;
// Synchronized to ensure thread-safe ID generation
public static synchronized String generateMedicineId() {
return "MED" + (++medicineCounter);
}
public static synchronized String generateCustomerId() {
return "CUS" + (++customerCounter);
}
public static synchronized String generateSupplierId() {
return "SUP" + (++supplierCounter);
}
public static synchronized String generateBillId() {
return "BIL" + (++billCounter);
}
public static synchronized String generatePrescriptionId() {
return "PRE" + (++prescriptionCounter);
}
// Update counters based on existing data
public static void updateCounters(int medCount, int cusCount, int supCount,
int bilCount, int preCount) {
if (medCount > medicineCounter) medicineCounter = medCount;
if (cusCount > customerCounter) customerCounter = cusCount;
if (supCount > supplierCounter) supplierCounter = supCount;
if (bilCount > billCounter) billCounter = bilCount;
if (preCount > prescriptionCounter) prescriptionCounter = preCount;
}
}
```

32. SampleDataLoader.java

The first time someone runs the application there is nothing in the database. Rather than staring at empty tables, this class fills in ten medicines spanning all eight categories, three customers, and three suppliers. It checks first whether medicines already exist - if they do, the whole method returns immediately so the demo data is never loaded twice.

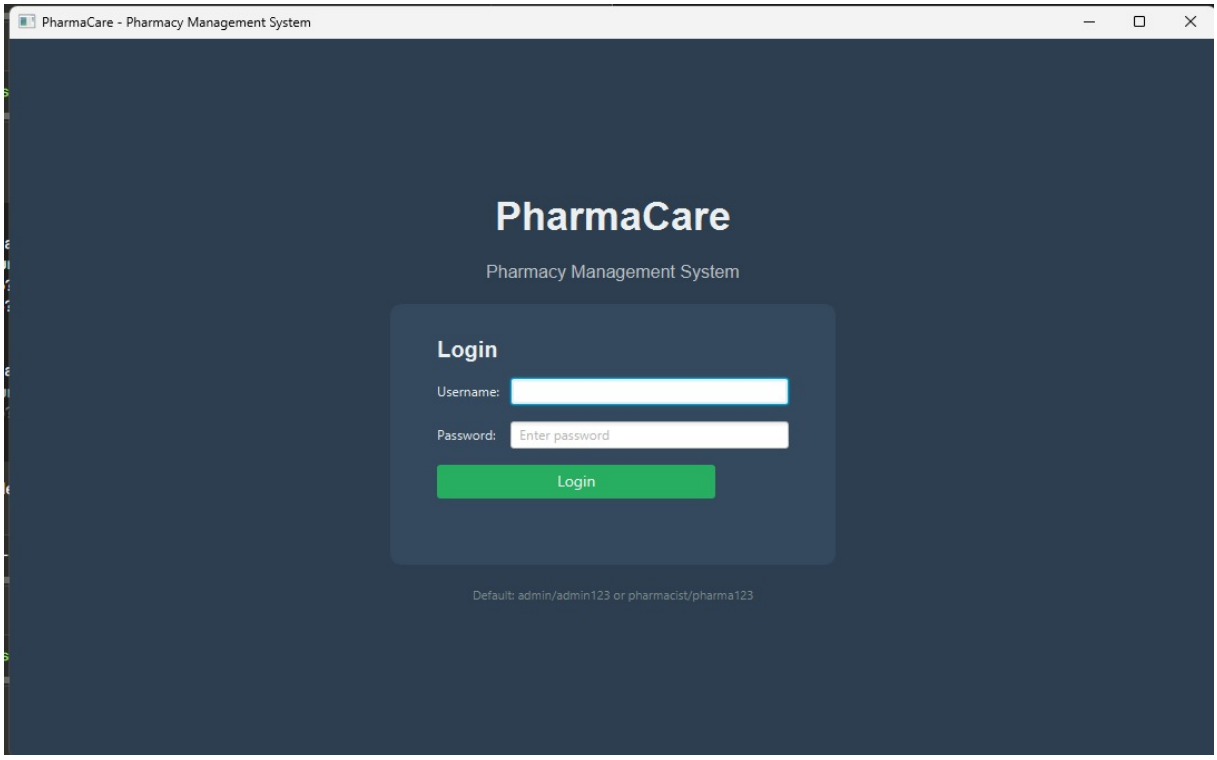
Code for SampleDataLoader.java:

```
package com.pharmacy.util;
import com.pharmacy.model.*;
import com.pharmacy.service.*;
/**
* Loads sample data for demonstration purposes.
* Demonstrates: Object creation, Enum usage, ArrayList operations.
*/
public class SampleDataLoader {
public static void loadSampleData(InventoryManager inventoryManager,
CustomerService customerService,
```

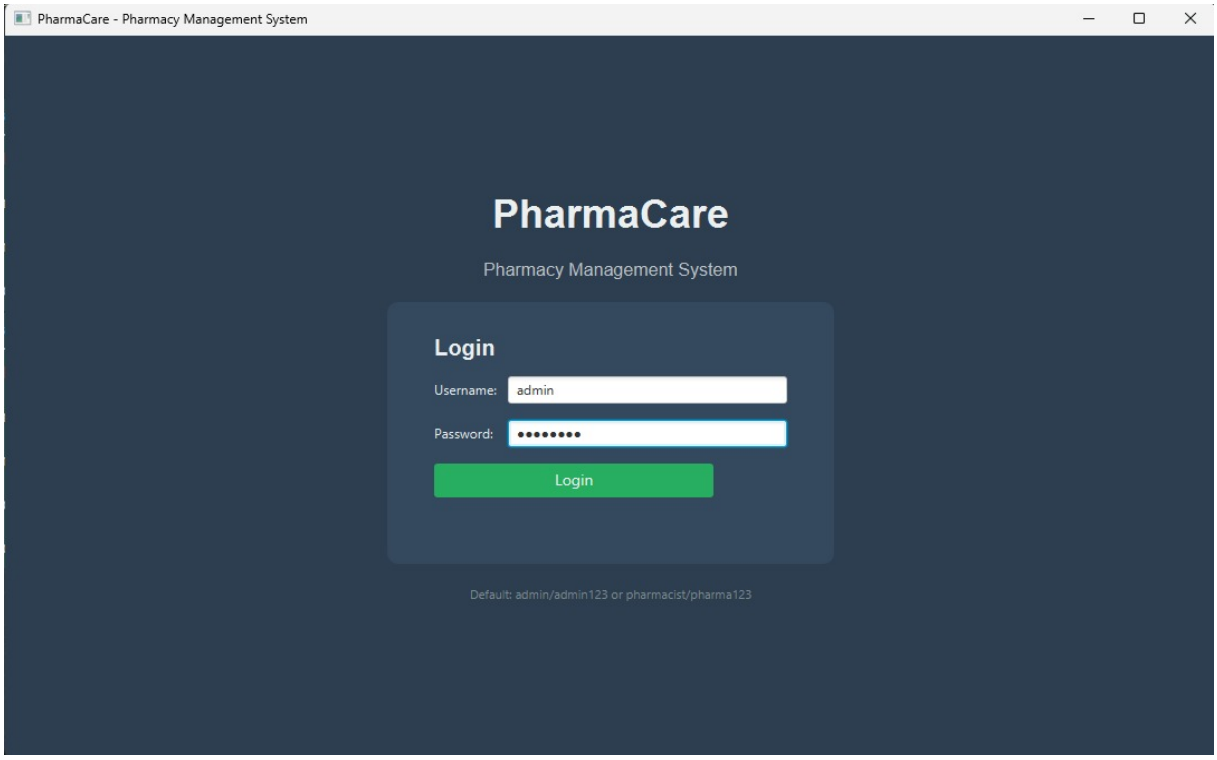
```
SupplierService supplierService) {
// Only load if no data exists
if (inventoryManager.getMedicineCount() > 0) {
return;
}
// Sample Suppliers
Supplier s1 = new Supplier("SUP3001", "PharmaCorp India", "Rajesh Kumar",
"9876543210", "rajesh@pharmacorp.com", "Mumbai, Maharashtra");
Supplier s2 = new Supplier("SUP3002", "MediLife Supplies", "Priya Sharma",
"9876543211", "priya@medilife.com", "Delhi, India");
Supplier s3 = new Supplier("SUP3003", "HealthFirst Pharma", "Amit Patel",
"9876543212", "amit@healthfirst.com", "Ahmedabad, Gujarat");
supplierService.addSupplier(s1);
supplierService.addSupplier(s2);
supplierService.addSupplier(s3);
// Sample Medicines
inventoryManager.addMedicine(new Medicine("MED1001", "Paracetamol 500mg", "Sun Pharma",
MedicineCategory.TABLET, "BT001", "2026-12-15", 500, 12.50, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1002", "Amoxicillin 250mg", "Cipla",
MedicineCategory.CAPSULE, "BT002", "2026-06-20", 200, 45.00, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1003", "Cough Syrup", "Himalaya",
MedicineCategory.SYRUP, "BT003", "2026-03-10", 150, 85.00, "SUP3002"));
inventoryManager.addMedicine(new Medicine("MED1004", "Insulin Injection", "Novo Nordisk",
MedicineCategory.INJECTION, "BT004", "2025-08-25", 50, 350.00, "SUP3003"));
inventoryManager.addMedicine(new Medicine("MED1005", "Dettol Ointment", "Reckitt",
MedicineCategory.OINTMENT, "BT005", "2027-01-30", 300, 65.00, "SUP3002"));
inventoryManager.addMedicine(new Medicine("MED1006", "Eye Drops", "Alcon",
MedicineCategory.DROPS, "BT006", "2026-09-15", 8, 120.00, "SUP3003"));
inventoryManager.addMedicine(new Medicine("MED1007", "Salbutamol Inhaler", "Cipla",
MedicineCategory.INHALER, "BT007", "2026-11-20", 5, 250.00, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1008", "ORS Powder", "WHO Standard",
MedicineCategory.POWDER, "BT008", "2027-06-10", 1000, 15.00, "SUP3002"));
inventoryManager.addMedicine(new Medicine("MED1009", "Azithromycin 500mg", "Zydus",
MedicineCategory.TABLET, "BT009", "2026-04-05", 100, 75.00, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1010", "Vitamin D3", "HealthVit",
MedicineCategory.CAPSULE, "BT010", "2026-10-30", 400, 30.00, "SUP3003"));
// Sample Customers
customerService.addCustomer(new Customer("CUS2001", "Rahul Verma", "9123456789",
"rahul@email.com", "Bangalore, Karnataka"));
customerService.addCustomer(new Customer("CUS2002", "Sneha Iyer", "9123456790",
"sneha@email.com", "Chennai, Tamil Nadu"));
customerService.addCustomer(new Customer("CUS2003", "Mohammed Ali", "9123456791",
"ali@email.com", "Hyderabad, Telangana"));
// Update ID counters
IDGenerator.updateCounters(1010, 2003, 3003, 4000, 5000);
}
}
```

Results

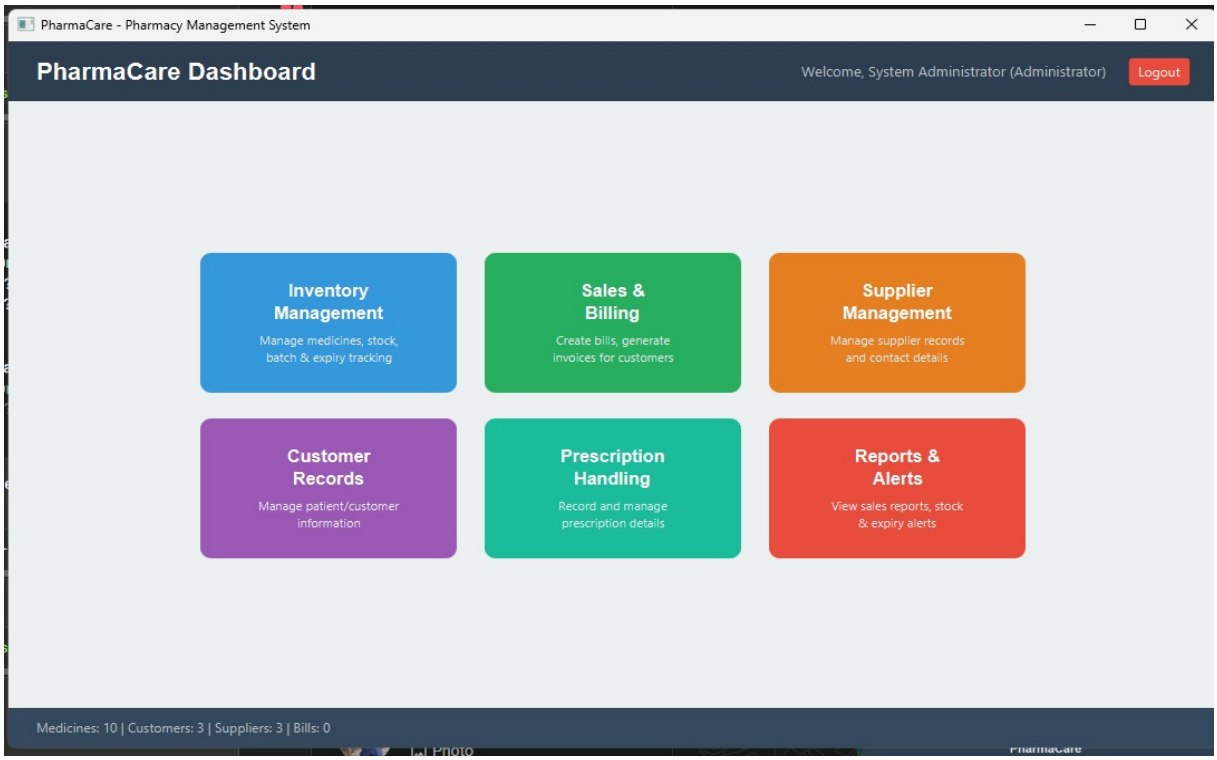
Login Screen



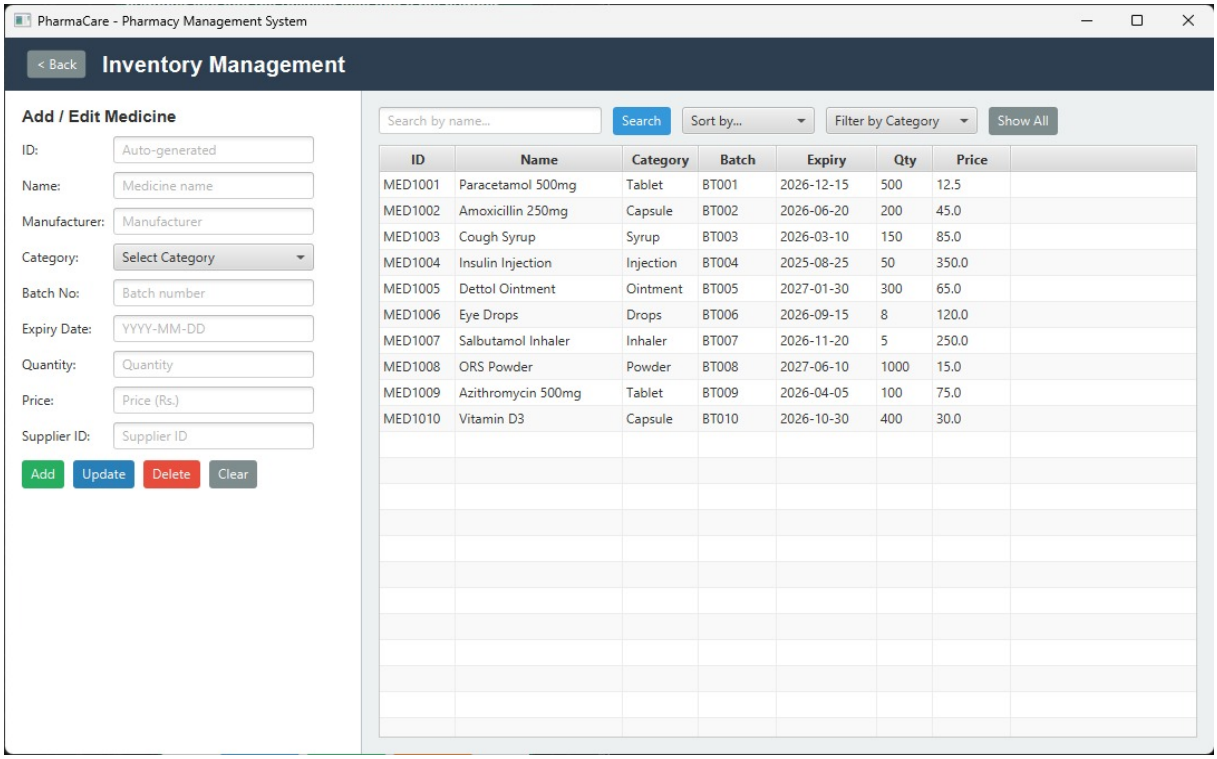
Login Screen - with Credentials



Dashboard Screen

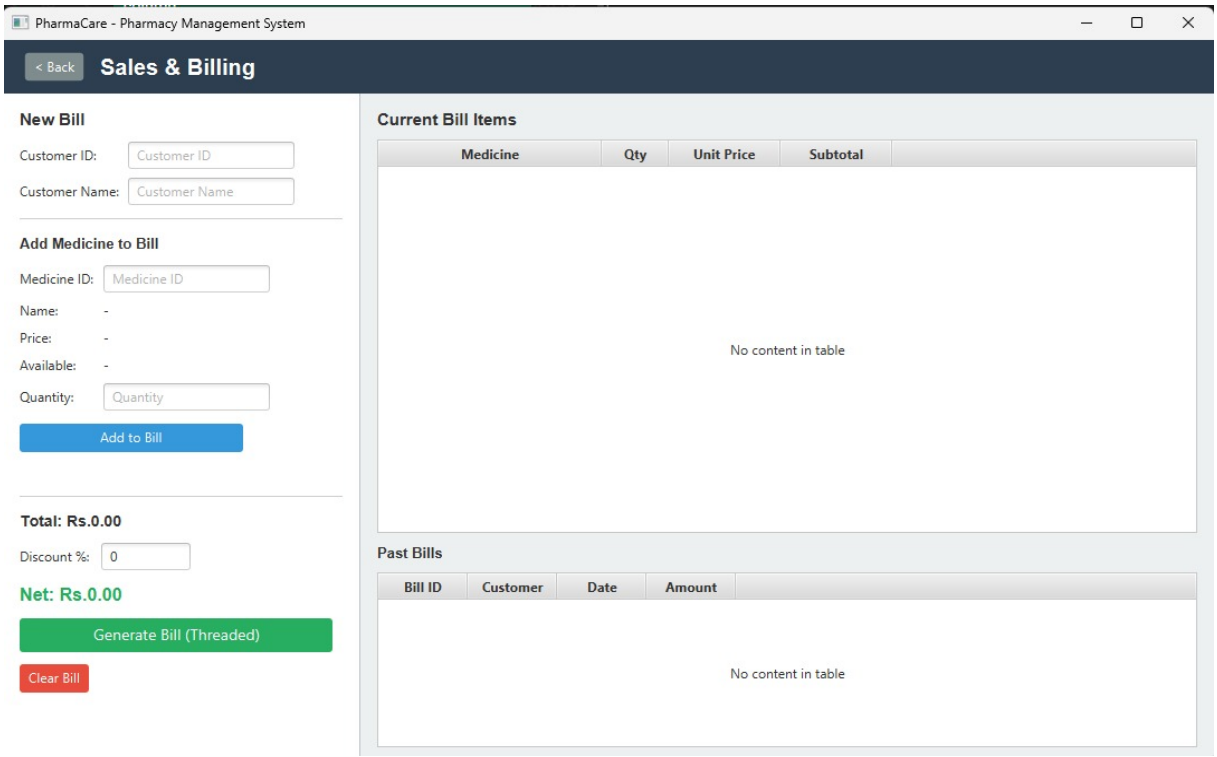


Inventory Management Screen

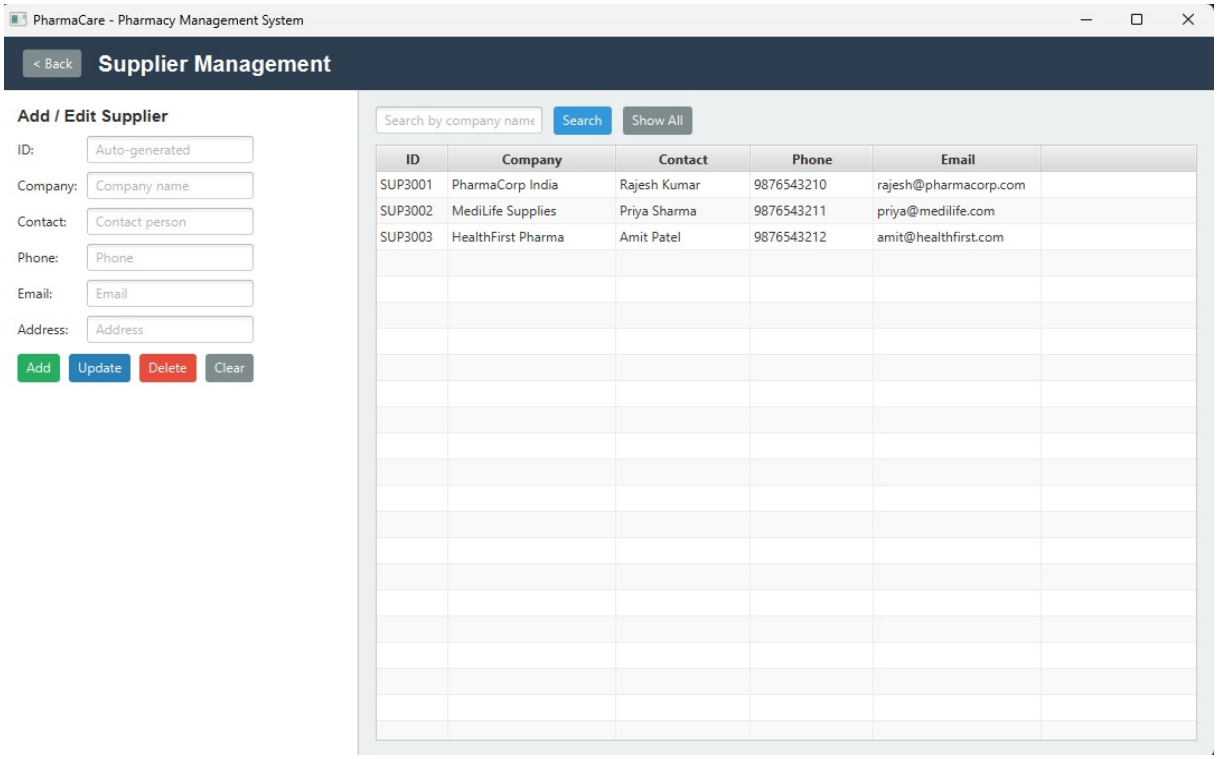


ID	Name	Category	Batch	Expiry	Qty	Price
MED1001	Paracetamol 500mg	Tablet	BT001	2026-12-15	500	12.5
MED1002	Amoxicillin 250mg	Capsule	BT002	2026-06-20	200	45.0
MED1003	Cough Syrup	Syrup	BT003	2026-03-10	150	85.0
MED1004	Insulin Injection	Injection	BT004	2025-08-25	50	350.0
MED1005	Dettol Ointment	Ointment	BT005	2027-01-30	300	65.0
MED1006	Eye Drops	Drops	BT006	2026-09-15	8	120.0
MED1007	Salbutamol Inhaler	Inhaler	BT007	2026-11-20	5	250.0
MED1008	ORS Powder	Powder	BT008	2027-06-10	1000	15.0
MED1009	Azithromycin 500mg	Tablet	BT009	2026-04-05	100	75.0
MED1010	Vitamin D3	Capsule	BT010	2026-10-30	400	30.0

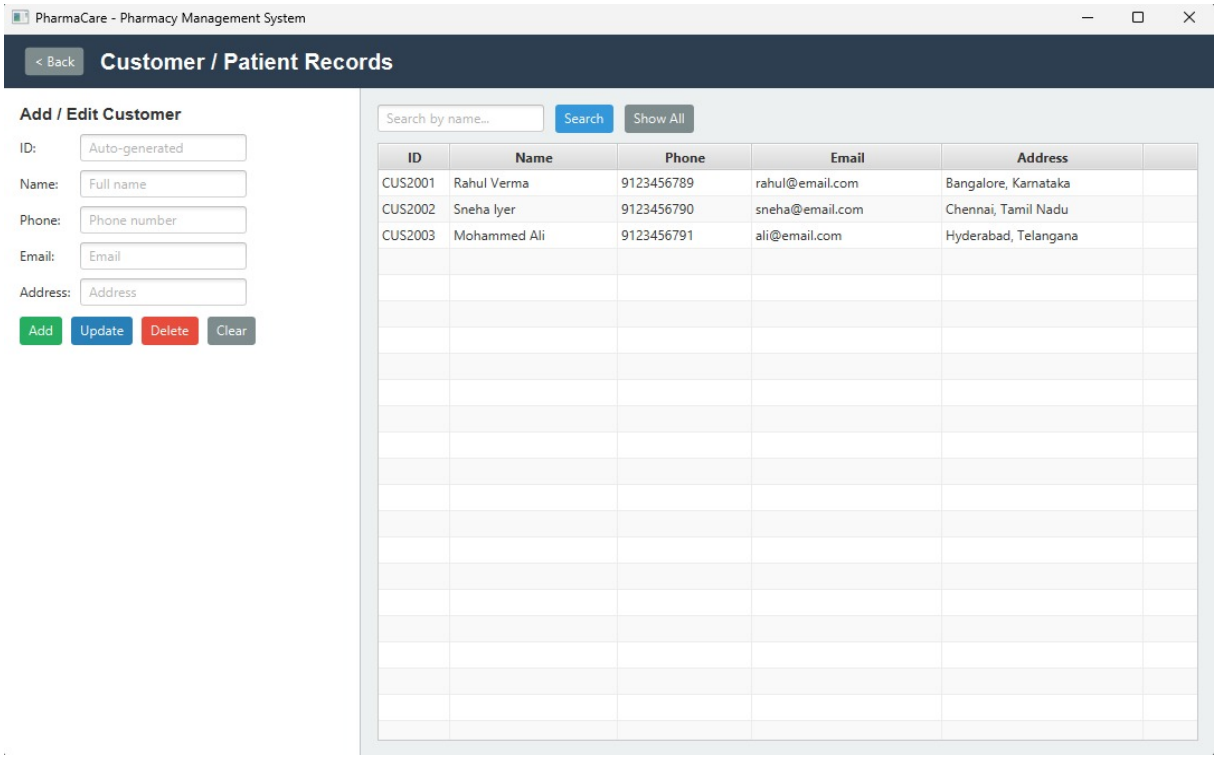
Sales and Billing Screen



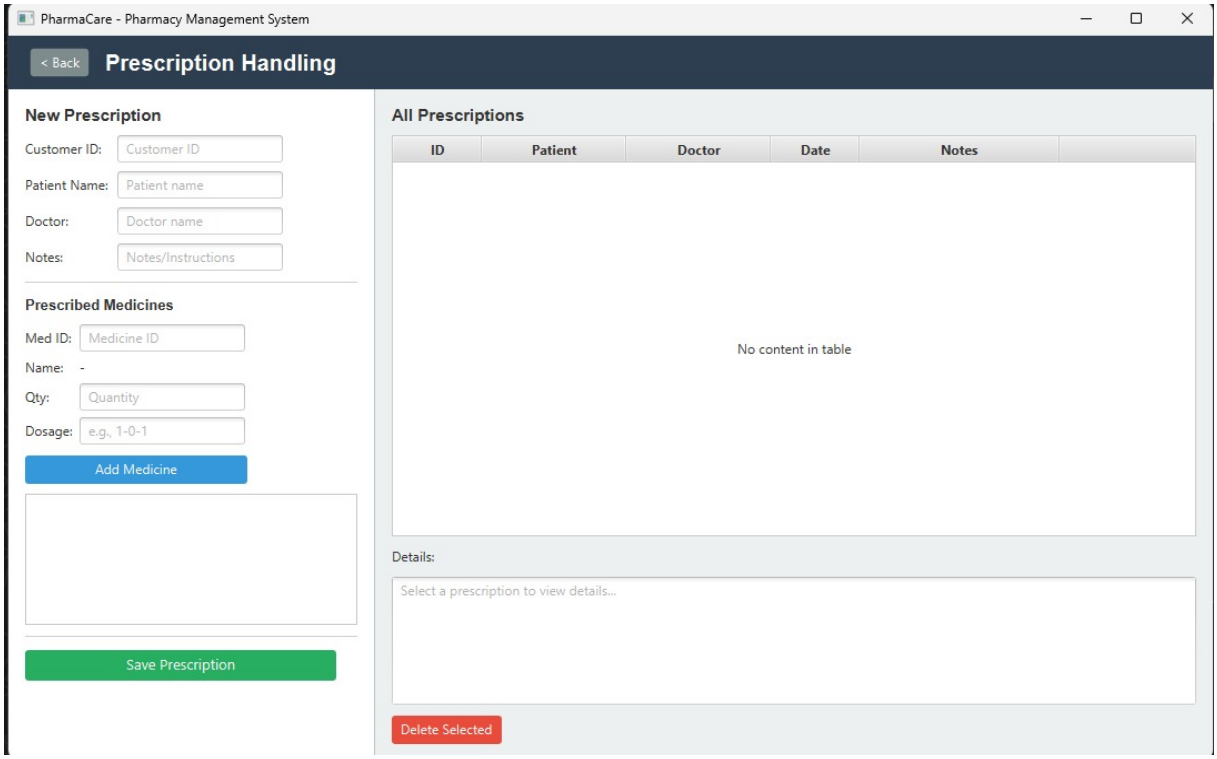
Supplier Management Screen



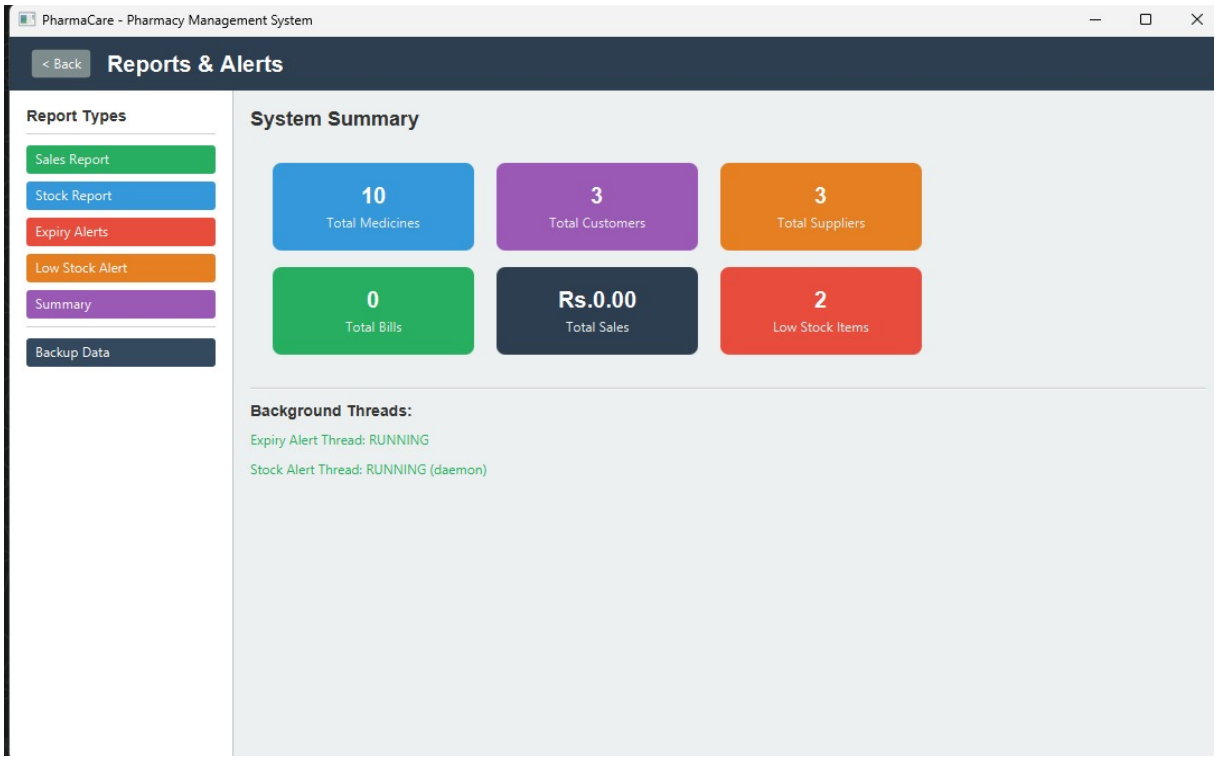
Customer Records Screen



Prescription Handling Screen



Reports and Alerts Screen


PharmaCare - Pharmacy Management System
< Back
Reports & Alerts
Report Types
Sales Report
Stock Report
Expiry Alerts
Low Stock Alert
Summary
Backup Data
System Summary
10
Total Medicines
3
Total Customers
3
Total Suppliers
0
Total Bills
Rs.0.00
Total Sales
2
Low Stock Items
Background Threads:
Expiry Alert Thread: RUNNING
Stock Alert Thread: RUNNING (daemon)

Conclusion

Getting the class structure right early on made an enormous difference to how the rest of the project went. The first version we sketched out had screen classes doing their own file operations, which quickly became hard to follow. Separating that out into proper service classes meant each file had one clear responsibility. After that split, adding the reports screen or the prescription module meant writing a new screen file and a new service file - nothing else needed to change.

The background thread work was something none of us had done in a full application before this project. Getting the expiry alerts to update the UI correctly took several tries - early attempts updated JavaFX components directly from the background thread, which caused intermittent crashes. The fix was to wrap those updates inside a call that schedules them back onto the main thread. Once we understood why that was needed, the rest of the threading work went much more smoothly.

Stepping back, this project touched every concept on the lab syllabus - but more importantly, each one appeared because the problem needed it, not because we were looking for a place to slot it in. Private fields with accessor methods kept data from being changed arbitrarily. The generic storage class eliminated four copies of identical file-handling code. The enum types caught category mismatches at compile time rather than runtime. Locks on the service methods protected data that two threads could otherwise corrupt simultaneously. The application ended up being a genuinely working piece of software, which made the whole exercise worth doing.

Individual Contributions

Kenneth Martin, 240962047, Roll No. 8

- PharmacyApp.java
- LoginScreen.java
- DashboardScreen.java
- UserService.java
- User.java / UserRole.java

Sam Thomas, 240962242, Roll No. 18

- Medicine.java / MedicineCategory.java
- InventoryManager.java
- InventoryScreen.java
- SampleDataLoader.java

Mohammed Rayyan Sheik, 240962073, Roll No. 12

- BillingService.java
- BillingThread.java
- BillingScreen.java
- Bill.java / BillItem.java

Vedank Singh, 240962043, Roll No. 7

- DataStore.java
- IDGenerator.java
- CustomerService.java / SupplierService.java
- PrescriptionService.java
- ExpiryAlertThread.java / StockAlertThread.java

- BackupService.java
- ReportsScreen.java
- Supplier.java / Customer.java / Prescription.java / PrescriptionItem.java / BillItem.java

The work was divided based on each person's strengths, but the nature of an OOP project means the parts are never truly independent. Kenneth's login flow needed the User and UserRole classes Sam had started. Rayyan's billing screen needed the DataStore and IDGenerator that Vedank was building. We spent a lot of time on a shared call resolving method signature mismatches and missing imports. That kind of back-and-forth might sound inefficient but it's actually what kept the four parts from drifting into four different styles - the final codebase reads like it was written by one person, which we consider the best outcome.

References

- 1. JavaFX Official Documentation: https://openjfx.io/
- 2. JavaFX Tutorials and Codes: https://www.geeksforgeeks.org/tag/javafx/
- 3. Java Serialization Guide: https://www.baeldung.com/java-serialization
- 4. Java Collections Framework: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html
- 5. Java Multithreading: https://www.geeksforgeeks.org/multithreading-in-java/